

Golden Thread Quest: System Design

How the local-first Golden Thread Quest application is designed and why, what is finished, and what remains. Written for a developer or technical stakeholder new to the system.

Version	2.2.0
Date	2026-09-29
Describes	main at be2b766, plus the Phase 2A changes landing with this version
Plan status	Gates 0 and 1 passed; Phase 2A (pilot release) current, Phase 2B (pilot run) next (Part 7, Section 7.2)
Source	docs/design/ (Markdown is the source of truth)

CONTENTS

1. About this document
2. Part 1. Purpose and users
 - 1.1 The problem
 - 1.2 Who uses it
 - 1.3 What the application is, and what it is not
 - 1.4 The system in context
 - 1.5 The ownership boundary
 - 1.6 How progress stays honest
3. Part 2. Architecture
 - 2.1 Two halves: a generator and a small service
 - 2.2 The components
 - 2.3 Content loading and JSON Schema validation
 - 2.4 View models and Jinja2 rendering
 - 2.5 The loopback action service
 - 2.6 The validator runner
 - 2.7 The progress store and locks
 - 2.8 The build pipeline and last-known-good output
 - 2.9 The update and migration path
4. Part 3. Data, identifiers and state
 - 3.1 The repository folders and who owns them
 - 3.2 Records and what each one holds
 - 3.3 Stable identifiers
 - 3.4 Versions and hashes
 - 3.5 The attempt state machine
5. Part 4. The system in motion
 - 4.1 How to read these diagrams
 - 4.2 A participant starts a quest and assembles evidence
 - 4.3 Running a validator
 - 4.4 Submitting for review
 - 4.5 The reviewer's decision
 - 4.6 The build and render pipeline
 - 4.7 Taking an upstream curriculum update
 - 4.8 One browser action through the loopback service

6. Part 5. The security model

- 5.1 Threats
- 5.2 The loopback boundary
- 5.3 No arbitrary command, no arbitrary path
- 5.4 Allowlisted writes
- 5.5 Untrusted content in the pages
- 5.6 Validator containment
- 5.7 Secret scan and redaction
- 5.8 Review integrity
- 5.9 Hostile repository files
- 5.10 External systems and privacy

7. Part 6. Key decisions

- 6.1 Product shape
- 6.2 Content and contracts
- 6.3 Generation and runtime
- 6.4 Safety and integrity
- 6.5 Concurrency and failure
- 6.6 Patterns across the decisions

8. Part 7. Where we are, and the phase plan

- 7.1 The goal
- 7.2 Where we are (2026-09-29)
- 7.3 How every phase runs
- 7.4 The phases
- 7.5 Where things are recorded
- 7.6 Known limitations of release one

9. Part 8. Glossary

About this document

About this document

This is the teaching design document for the Golden Thread Quest application. It is written for a developer or technical stakeholder who has never seen the system, and it has two jobs:

1. explain what the system is for and how it is designed, in order, with the reason for each design choice;
2. say where the work stands and how the rest of it is planned: the goal, the phases, and the gate between each phase where scope may change.

It describes the code at the commit above. Where the code and the specification documents

(`docs/ARCHITECTURE.md`, `docs/CONTENT-MODEL.md` and others) disagree, this document follows the code and says so at the point of disagreement. Where an open audit finding shows the code falling short of the design, the finding is named beside the claim.

The Markdown files in this folder are the source of truth. The HTML and PDF are rendered from them and carry the same version in their file names and on their title page. `REGENERATING.md` explains the version rule and how to rebuild both.

Conventions

- Every diagram is Mermaid, drawn from the code, with a numbered walkthrough beside it. GitHub renders the Mermaid blocks; the HTML carries them as inline SVG.
- `ADR-nnn` refers to `docs/DECISIONS.md`. Audit finding IDs (C1, E9 and so on) refer to the round record named beside them under `docs/audits/`. `Dn` refers to the deferred-work register in `docs/IMPLEMENTATION-PLAN.md`.
- Commands are run from the repository root. `quest-app` needs the virtual environment active: `make setup`, then `source .venv/bin/activate`.
- Terms are defined at first use and collected in the glossary.

Part 1. Purpose and users

This part explains the problem the application solves, who uses it, and the one boundary that shapes every later design choice: who owns which files.

1.1 The problem

The Golden Thread Quest trains an **AI Context Engineer**: a person who owns the two ends of software delivery that agentic tools touch most.

- The **intent bookend**: capturing conversations, decisions and requirements, structuring them as local context, and turning them into actionable work items.
- The **validation bookend**: turning that intent into tests, running them, keeping evidence, and saying what risk remains.

The "golden thread" is the traceable line between the two:

Conversation → requirement → work item → implementation → test → evidence → outcome

Training for this role has a specific difficulty. The skills are practical, so a quiz proves little. The work happens in real repositories with real tools, so a hosted course cannot see it. And the most important question a mentor asks, "did this person actually do it, and does it hold up?", needs a human judgment backed by evidence, not a checkbox.

The application answers that difficulty with three commitments:

1. **The curriculum is content.** Each unit of work is a **quest**: one Markdown file with YAML front matter that states the mission, acceptance criteria and required proof. Adding a quest needs no code change.
2. **The participant's work is ordinary files in their own Git repository.** The application reads those files, checks them and presents them. It does not own them and does not hold them in a database.
3. **Progress stays honest.** What a participant claims, what an automated check establishes and what a reviewer approves are different states with different authorities. Only a reviewer's approval produces *verified* completion.

1.2 Who uses it

Role	Who they are	What they need from the application
Participant	A business analyst, tester, Scrum facilitator, QA professional or developer learning agentic workflows, usually working alone and at their own pace	Know the next useful quest, understand what success means without hidden knowledge, build real artifacts, assemble evidence, resume after an interruption, and see the difference between "I finished" and "a reviewer confirmed it"
Reviewer (also called <i>Questmaster</i>)	A facilitator, mentor or peer who evaluates evidence	See what the quest asked for, what was submitted and what the checks found; reproduce it; approve, or ask for changes with reasons; never handle the participant's secrets
Curriculum maintainer (curriculum author)	The program owner who writes quests, regions, badges and validators	Add or change curriculum through content files alone, and get an actionable error when content is wrong

A fourth actor is not a person: the **upstream repository**, the canonical copy of the program from which participants take curriculum updates (ADR-002).

1.3 What the application is, and what it is not

It **is** a local-first, repository-native training application with two halves:

- a **generator** that turns validated content and participant state into a static HTML site (ADR-006, ADR-008), and
- a small **loopback service**, an HTTP server bound to `127.0.0.1`, that performs the few things a static page cannot: record a state change, run a registered check, rebuild (ADR-008, ADR-022). A command-line surface, `quest-app action`, performs the same actions through the same code for places with no browser, such as an agent sandbox.

It **is not** a learning-management system, a hosted multi-user service, a central database, a leaderboard, or a tool that writes to Jira, Trello or GitHub on the participant's behalf. Release one performs no external write at all. These exclusions come from `PLANNING-STATUS.md` and are enforced in code, not only stated.

1.4 The system in context

Walkthrough. Read the figure from the top.

1. The **curriculum maintainer** commits quests, regions, badges, tracks and validator code to the upstream repository. That content flows into each participant's fork by an ordinary Git merge that the participant runs.
2. The **participant** works in their fork. Their files live under `participant/`. They drive the application through a browser pointed at the loopback service, or through the `quest-app` command line.
3. The **reviewer** reads the participant's evidence, usually from a pull request or a checkout of the participant's repository, and records a decision through the same application. The decision is a file in the participant's evidence directory.
4. The application reads authored content and participant files, validates both, and writes generated HTML under `generated/`. It writes participant files only through a short allowlist of actions.
5. The application makes **no outbound network request of its own**. External systems (Jira, Trello, GitHub) are things the participant's own scripts talk to while doing a quest; the application never does.

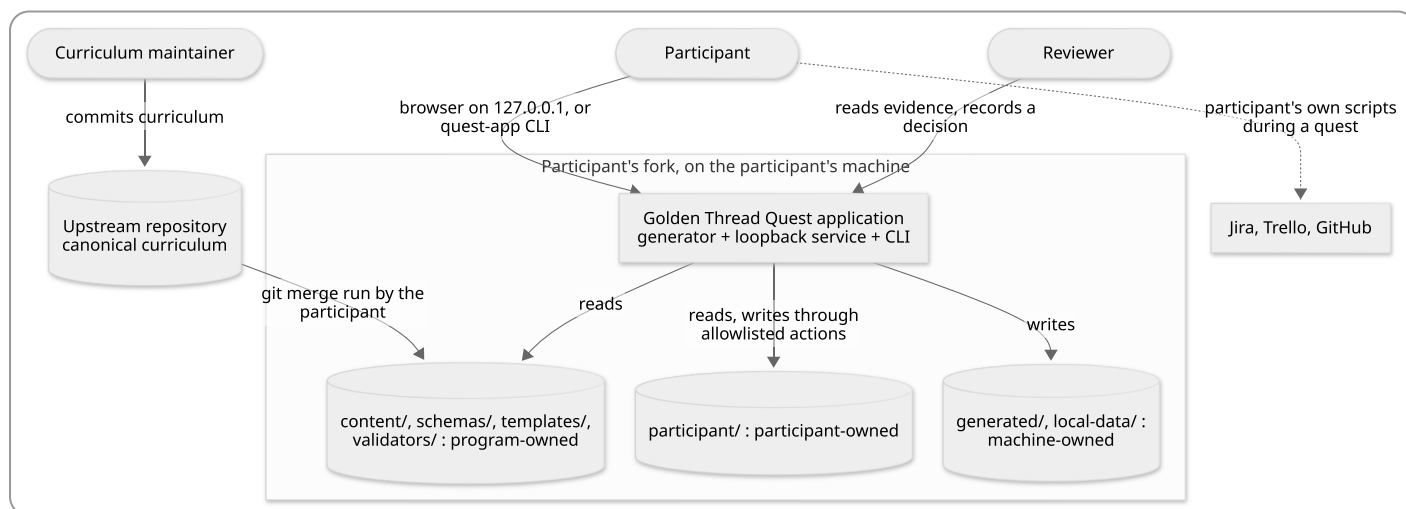


Figure 1. System context. Solid arrows are what the application does; the dotted arrow is work the participant does outside it.

1.5 The ownership boundary

Every file in the repository belongs to exactly one of three owners (ADR-003). This is the most important idea in the design, because the rest of the architecture exists to keep the three apart.

Zone	Folders	Owner	Who may change it	What happens on an upstream update
Program-owned	content/ , schemas/ , templates/ , assets/ , quest_app/ , validators/ , most of docs/	The program (curriculum maintainer and application developers)	Upstream, by commit	Merged in
Participant-owned	participant/	The participant	The participant in any editor, and the application through a narrow set of actions	Never replaced
Machine-owned	generated/ , local-data/	Nobody: disposable	The application	Irrelevant: Git ignores them and a build recreates them

Three consequences follow, and each appears again later:

- **Updates cannot destroy work.** Because the participant's files sit in their own folder, taking new curriculum is an ordinary merge that touches only program-owned folders (Part 4, flow 4.7).
- **Generated output is never the truth.** A deleted or corrupt `generated/` directory is fixed by running a build (ADR-015). Canonical truth is content, participant files and review records.
- **The participant's files are untrusted input.** The participant can edit `participant/progress.yaml` by hand. The application therefore re-derives anything the participant is not allowed to decide, above all `verified`, from the review record rather than believing the file (ADR-011, Part 3).

Part 3 draws the folders in detail. Part 5 explains how writes are confined to the participant zone.

1.6 How progress stays honest

The application separates four kinds of statement about a quest, each with its own authority:

Statement	Authority	Example
"I have started" / "My evidence is ready"	The participant	<code>in_progress</code> , <code>evidence_ready</code>
"The automated checks pass"	A registered validator, requested by the participant	<code>locally_validated</code>
"Please review this"	The participant	<code>submitted</code>
"This is verified" / "This needs changes"	A reviewer	<code>verified</code> , <code>needs_changes</code>

Two XP totals follow from this and are never added together:

- **Claimed XP** counts quests from `evidence_ready` onward. It is the participant's own record.

- **Verified XP** counts only quests with an approved review. Nothing a participant does and nothing a validator returns can produce it (ADR-011).

The **proof hierarchy** says what counts as good evidence, strongest first: a repository artifact, reproducible instructions, automated execution evidence, a peer demonstration, and reviewer approval. Screenshots are supporting evidence only (ADR-013).

Part 2. Architecture

This part names the components, explains what each one is responsible for, and says why it is shaped that way. Part 4 then walks through the same components in motion.

2.1 Two halves: a generator and a small service

The application is a **hybrid** (ADR-008):

- **Static generation.** Python loads the content and the participant's state, validates both, computes every derived fact (which quests are locked, what XP is claimed or verified, which badges are earned), and renders HTML pages with Jinja2 into `generated/`. The pages are ordinary files: they open from disk with no server.
- **A loopback action service.** A small HTTP server on `127.0.0.1` serves those pages and accepts a short list of actions: start a quest, mark evidence ready, run a validator, submit, record a review, rebuild. After each action it rebuilds the site so the pages show the new state.

Why this split. Pure static HTML cannot create files, run checks or read Git, so some process has to. A full dynamic web application would duplicate state between a server and a browser and widen the attack surface. The hybrid keeps everything that is read as static, deterministic output and everything that writes behind one narrow, checkable door.

A third surface, the **command line** (`quest-app`), calls the same action code directly. It exists because the primary target surface, an agent sandbox such as the Cowork app, has a filesystem but cannot reach a loopback port from the user's browser.

2.2 The components

Walkthrough. The figure groups modules by responsibility. Read it top to bottom.

1. **Surfaces.** Three entry points: the CLI (`cli.py`), the loopback service (`serve.py`), and the static pages themselves. The CLI and the service both call the **action layer**; neither holds any rule of its own about what a participant may do (ADR-033).
2. **Action layer.** `actions.py` holds `ActionRunner`, the single allowlist of mutating actions and every guard: prerequisites, confirmation, secret scan, qualifying validation. `state_machine.py` holds the transition table and the confirmation texts.
3. **Loading and validation.** `pipeline.load_world` is the one way to obtain a consistent picture of the world: it calls `content_loader.py` (discover, parse, schema-validate, normalize), `semantics.py` (cross-document rules), `validator_registry.py` (validator references), and `progress.py` (participant state, read but never believed).
4. **Derivation and presentation.** `progress_calc.py` computes states, XP and badges; `recommend.py` picks the next quest; `view_models.py` turns all of it into display-ready objects; `build.py` renders templates, writes indexes and publishes atomically.
5. **Writing and running.** `store.py` writes participant files atomically under a lock; `review.py` writes submission and review records; `evidence.py` detects proof, scans for secrets and hashes evidence; `validator_runner.py` runs a validator in a child process (`validator_child.py`).
6. **Repository tools.** `git_status.py` runs five read-only Git commands; `update.py` and `migrations.py` handle upstream updates.

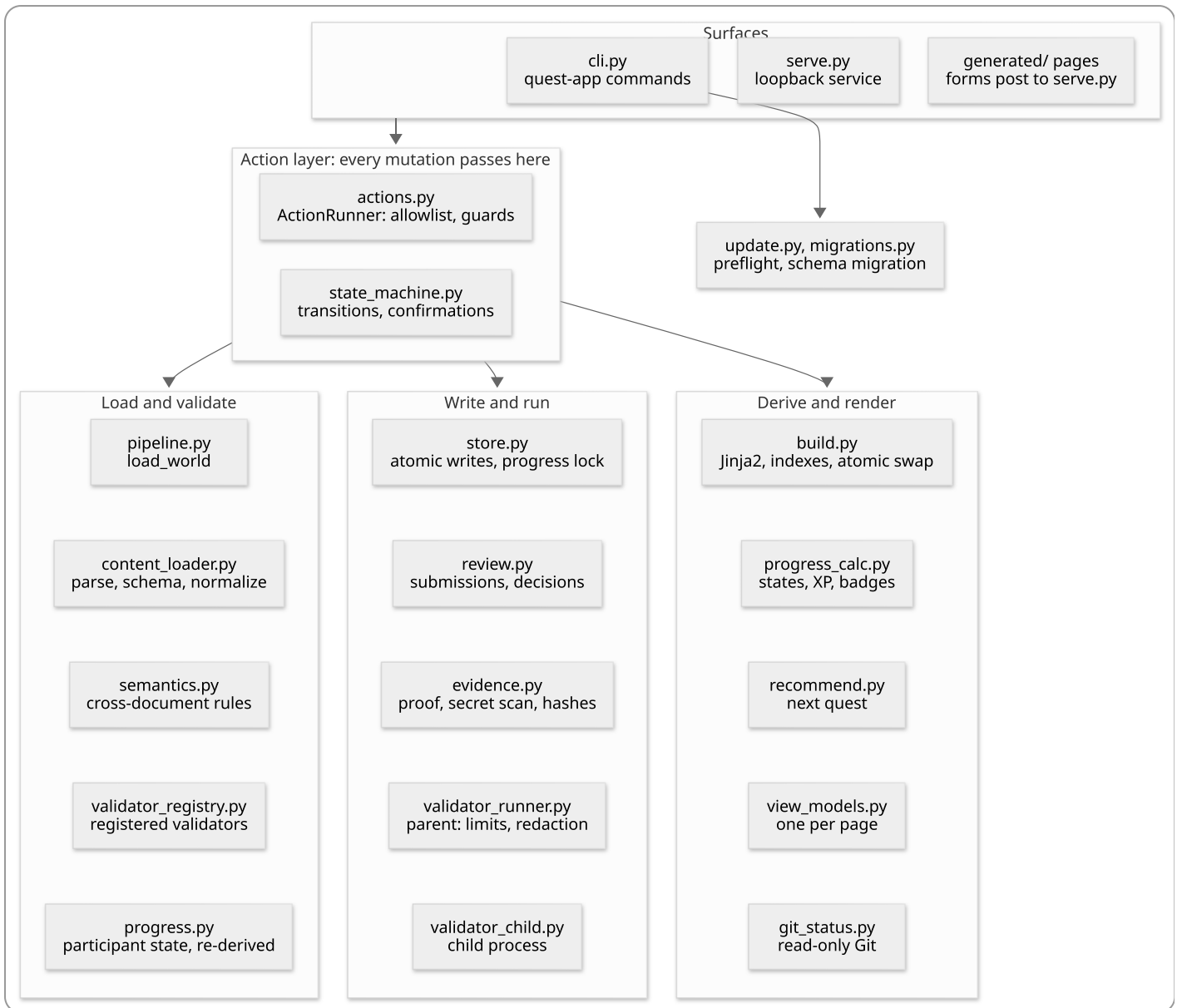


Figure 2. Components as implemented in `quest_app/`. Arrows run between groups: the action layer calls into all three lower groups, and `build.py` writes the pages the surfaces serve. `docs/ARCHITECTURE.md` names a single `progress_manager.py`; the code splits that role across `progress.py` (read), `store.py` (write), `progress_calc.py` (derive), `state_machine.py` and `actions.py`.

2.3 Content loading and JSON Schema validation

What happens. `content_loader.load_content` runs the same four steps every time:

1. **Discover** files under `content/` (quests are `content/quests/<region>/<name>.md`; regions, badges and tracks are YAML; `content/site.yaml` holds site identity). Only regular files inside the content tree are read, so a symbolic link to a device or a FIFO cannot hang the loader.
2. **Parse** YAML with a strict safe loader (`yaml_loader.StrictSafeLoader`): safe constructors only (ADR-025), duplicate keys refused (ADR-029), deep nesting reported as an ordinary problem.
3. **Validate** each document against its JSON Schema in `schemas/` using `jsonschema` Draft 2020-12 (ADR-005, ADR-020). There are ten schemas: quest, region, badge, track, site, progress, submission, review, validation-result and validator-registry.
4. **Normalize** each valid document into a frozen dataclass (`models.py`). Nothing downstream sees a raw dictionary. Quest Markdown is rendered with `markdown-it-py`, raw HTML disabled, then sanitized with `nh3` (ADR-021); the result is carried in a field named `safe_rendered_html` so it cannot be confused with plain text.

`semantics.validate_bundle` then applies the rules a schema cannot express, because they span documents: duplicate IDs, unknown region, prerequisite, badge or related-quest references, prerequisite cycles, a quest whose proof names a validator it does not declare, empty regions, unreachable badges, and more. Some are errors that stop the build; others, such as an unusual XP value for the chosen level, are warnings.

Why schemas are the contract. A JSON Schema is language-neutral, editor-friendly and readable by a maintainer who does not read Python. A second model layer (for example Pydantic) would restate the same rules in a place that can drift from the file the maintainer reads (ADR-020).

Why errors are shaped as they are. Every failure is a `ContentProblem` naming the file, the stable ID when known, the field path, the rule in words, a redacted received value, and a suggestion (for a mistyped ID, the closest real one). The person fixing it is a curriculum author in a text editor, not a developer reading a traceback.

Acceptance criteria. A quest's criteria are not duplicated into front matter. They are read from the ordered list under the body's `## Acceptance criteria` heading, from the Markdown token stream rather than by line matching (ADR-027), and each gets the positional ID `ac-<n>` plus a hash of its text (ADR-016, amended by ADR-026). A reviewer's finding can then point at `ac-3`, and the page and the finding agree on what `ac-3` means.

2.4 View models and Jinja2 rendering

What happens. `build.build_site` takes a `LoadedWorld` and:

1. computes per-quest state with `progress_calc.compute_states` (Part 3 explains the rule), region progress, claimed and verified totals, badge progress, and recommendations;
2. builds one view model per page in `view_models.py` (for example the quest summary, the evidence workspace, the reviewer view), each a display-ready object;
3. renders the thirteen page templates in `templates/pages/` with Jinja2, autoescaping on and `StrictUndefined` so a missing field is an error rather than an empty string;
4. writes search, tag, region, relationship and state indexes as JSON under `generated/indexes/`, plus `build-manifest.json` with the application version and the content hashes;
5. publishes the whole site atomically (Section 2.8).

The eleven screens specified in `docs/ui/SCREEN-SPECS.md` (U01 Home to U11 Content Author Error) map to thirteen templates: the extra two are the tag page and the evidence index.

Why view models. Templates contain no quest-specific content and no logic that decides anything (`docs/VIEW-MODEL-CONTRACT.md`). They do not open files, calculate progress, resolve prerequisites or decide authority. That keeps one place, Python, responsible for every rule, and lets a test assert that adding a quest leaves `templates/`, `assets/` and `quest_app/` byte-identical.

Why deterministic. Traversals are sorted and JSON is written with sorted keys, so a diff of `generated/` means content changed. The one varying value is the build time on each page. Set `SOURCE_DATE_EPOCH` and two builds of the same inputs are byte-identical (ADR-040).

2.5 The loopback action service

What it is. `serve.py` runs `http.server.ThreadingHTTPServer` from the Python standard library with an explicit route table (ADR-022). It serves `generated/` and exposes:

Route	Method	Purpose
<code>/api/health</code>	GET	Application and service status
<code>/api/git-status</code>	GET	A summary from the five read-only Git commands

Route	Method	Purpose
<code>/api/action</code>	POST, JSON body	Perform one allowlisted action
<code>/api/action/<action>/<quest-id>[/<validator-id>]</code>	POST, HTML form	The same, from a page with no JavaScript
anything else	GET	A static file from <code>generated/</code> , with the request token substituted into HTML

Why the standard library. Every requirement on the service is a *restriction*: bind to loopback, require a token, check origin, cap the body, accept only allowlisted IDs. A framework adds dependencies and hides the exact request surface those restrictions apply to. One local user needs no concurrency model beyond threads.

The request token. At startup the service mints a random token (`secrets.token_urlsafe(32)`), keeps it in memory and never writes it to disk or prints it. Built pages contain a placeholder; the service substitutes the token as it serves each HTML page, so a form on a page this run served carries it. A page opened from disk, or served by another run, does not have it, and its forms are refused.

Which port is running. The service writes the port it bound to a file per port under `local-data/service-ports/` and removes it on shutdown (ADR-037). A CLI action or build probes those ports, and believes an answer only if it carries this application's response header and repository signature. It needs to know, because a rebuilt page says whether its controls work: pages built while a service runs have live forms; pages built without one say "Start the local service".

Part 5 lists every check a request meets. Part 4, flow 4.8, shows one request end to end.

2.6 The validator runner

What a validator is. A **validator** is a program-owned Python function that checks a participant's artifacts and returns structured findings. Each is registered by stable ID in `validators/registry.yaml` with an entrypoint (`validators.<module>:run`), a working directory, read and write roots, an environment-variable allowlist, a timeout, an output limit, the quests it may run for, and a `network: denied` declaration. A quest names a validator by ID and can never supply a command, argument or path (ADR-014).

Release one registers three quest-facing validators (`validate-repository-foundation`, `validate-jira-read-assigned`, `validate-playwright-quality`) and two environment probes used by tests.

How a run is isolated. `validator_runner.run_validator`:

- starts a fresh interpreter as a child process with a fixed argument list and no shell, in its own session and process group (`start_new_session=True`);
- builds the child's environment from a minimal `PATH` plus the registry's allowlist, never inheriting the parent's environment, so an exported credential does not leak into a check;
- puts the repository first on the child's import path, so a `json.py` in the participant's folder cannot replace a standard module or the registered code;
- gives the validator a `Workspace` object whose read and write methods resolve symbolic links and re-check that the path is inside the registered roots, and that exposes the evidence package of *the attempt under validation* (ADR-039);
- reads output as it arrives, stops a run that writes more than 1 MiB on a stream, kills the whole process group on timeout, on exit, or a second after the child's result channel (its `stdout`) has closed while the OS process is still alive — a non-daemon thread the validator forgot to stop no longer costs the run its result (round 12, finding E11);

- redacts the complete output and every check field *before* truncating them, never after: cutting first can sever a secret-shaped token at the boundary, and the half that survives matches no detector (round 12, finding E7).

Every check a validator's process reports is also checked against the result schema's own `id` pattern and `outcome` enum before it is trusted: a check outside either is replaced with one reporting that a check was malformed, and the run's outcome is forced to `environment_failure`, because a validator that cannot describe its own check correctly has a defect in itself, not a fact about the participant's work — the same principle already applied to an uncaught exception or a nonzero exit (round 12, finding E6).

Outcomes. A run ends in one of six outcomes: `pass`, `fail`, `warning`, `environment_failure`, `inconclusive`, `interrupted`. Only `pass` and `warning` are **qualifying** results. Part 4, flow 4.3, shows how each outcome is reached.

A run is not a transition. Running a validator appends a result file and changes nothing else (ADR-017). The participant then asks for `locally_validated`, and that request succeeds only if every declared validator's latest run qualifies. A validator is therefore a source of facts, never an authority over the participant's state.

What this is not. The controls bound a validator through its `Workspace` and its process. They are not an operating-system sandbox: `network: denied` is a declaration nothing enforces, a validator calling `open()` directly bypasses the roots, and a grandchild that starts its own session survives cleanup (`docs/VALIDATOR-CONTRACT.md`; deferred item D9). Validators are program-owned code, reviewed like the rest of the application.

2.7 The progress store and locks

The store. `store.ProgressStore` is the only writer of `participant/progress.yaml`. Every write:

1. validates the new document against `schemas/progress.schema.json` *before* writing, so the application never writes a file it would later refuse to load;
2. writes to a temporary file in the same directory, flushes and `fsync`s it, then renames it over the original (`atomic_write_text`, which calls `safe_io.atomic_write`). A rename within one filesystem is atomic, so an interrupted write leaves the old file intact;
3. appends a line to `participant/ACTIVITY.md` describing the change, so the participant can read everything the application did to their files.

Every participant write follows the same rule, not just this one (ADR-042). Round 12 found that the activity line was appended with a plain `open("a")`, which follows a symbolic link wherever it leads and blocks forever on a FIFO; that validation results were written into a `validation/` directory that could itself be a link out of the tree; and that the review archive used a plain `write_text`. `safe_io.atomic_write` and `safe_io.append_to_regular_file` now back every one of these: each walks from the participant root one component at a time with `O_NOFOLLOW`, refuses a component that is a link or not a directory, refuses a target that exists and is not a regular file, and never opens anything in a way that can block. When the unusable target is `ACTIVITY.md`, the line is skipped and a warning goes to stderr, because by the time it runs the change it describes is already on disk in `progress.yaml`; refusing the action at that point, or reporting a failure that did not happen, would cost the participant's trust in their own record for less reason than skipping one note. For every other target — `progress.yaml`, a validation result, a submission or review record — the write is refused before anything changes, like any other write failure.

Configuration, not just the participant root. `GTQ_PARTICIPANT_ROOT`, `GTQ_GENERATED_ROOT` and `GTQ_LOCAL_DATA_ROOT` are all read by `AppConfig.from_environment`, and `_config_from_args` carries all three forward when `--participant-root` is passed on the command line (ADR-018, amended round 12). Before the amendment, only the participant root moved: a test that ran the CLI as a real subprocess — the only way an installed sandbox participant can act at all — rebuilt the repository's own `generated/` on every mutating action, overwriting a checked-in page with fixture data until the next `make build`. A session-scoped test fixture

(`tests/conftest.py`) now fails the whole run if the repository's own `generated/`, `local-data/` or `participant/` changed anyway, as the backstop for a test that forgets to set one of the three.

Three locks. Each one protects a different race:

Lock	Where	Protects	Decision
Service lock	A <code>threading.Lock</code> in the service process	Two browser requests interleaving inside one service	<code>serve.py</code> ; context in ADR-034
Progress lock	<code>participant/.progress.lock</code> , an exclusive POSIX <code>flock</code> , held by <code>ActionRunner.perform</code> across load, decision and write	A CLI action and a service action (two processes) interleaving a read-modify-write	ADR-034
Build lock	<code>generated.lock</code> , an exclusive <code>flock</code> held for the whole build	Two builds sharing the fixed staging directory <code>generated.building</code>	ADR-035

The progress lock is held across the *load* as well as the write, because reading state another process is about to replace is the race. It is not held over the validator run, which is slow and only appends.

Why a lock failure never refuses the work. Where a lock cannot be taken (no `fcntl` module, an unopenable lock file, or a filesystem that answers `ENOLCK`, such as some network home directories), the work proceeds unlocked (ADR-036). The atomic rename still guarantees the file is never half-written, and refusing to record a participant's own work because of a lock file is worse than the race.

Why a failed rebuild does not fail the action. An action writes its change, then rebuilds. If the rebuild fails, the action is still reported as done, with an advisory that the site could not be rebuilt (ADR-038). The participant's record cannot be regenerated; the site can, by one command.

2.8 The build pipeline and last-known-good output

A build renders the entire site into a sibling directory, `generated.building`, and then publishes by renaming: the current `generated/` becomes `generated.previous`, the staged directory becomes `generated/`, and the previous copy is removed. Until that rename, a reader keeps seeing the old site. If content fails validation, nothing is rendered into `generated/` at all: the errors are written as a page to `local-data/build-errors/index.html` and the last good site stays in place.

`docs/ARCHITECTURE.md` lists "copy fingerprinted or versioned static assets" as a build step. The code copies `assets/` as-is; `routes.asset` accepts a fingerprint but no caller passes one.

Part 4, flow 4.6, draws the pipeline.

2.9 The update and migration path

A participant's fork has two remotes: `origin` (their fork) and `upstream` (the canonical repository). Taking new curriculum is a Git merge from `upstream`.

The application never runs the merge. `quest-app update` (also `make update-check`) is a **preflight**: it checks that the directory is a Git repository, that no merge is in progress, that the working tree is clean, that an `upstream` remote exists, and that the participant folder is present. It reports whether a schema migration would be needed and which attempts are on an older quest version, proposes a backup branch name, prints the exact commands, and stops. Every Git command `update.py` may run is read-only.

Migrations. `quest-app update --migrate` (also `make migrate`) moves `participant/progress.yaml` to the schema version this build reads. It runs under the progress lock, validates before and after, reloads the whole

world after writing, and restores the original bytes if that reload fails. Two rules apply to every migration: an in-progress attempt's quest version is never advanced, and an unknown field is carried forward rather than dropped. Release one ships schema version 1, so the only step is the identity step; the machinery exists so the safety is designed before the first real migration.

Where this differs from the plan. `docs/ARCHITECTURE.md` describes a future helper that "may create a backup branch, merge or rebase, apply migrations". The code does none of the Git writes: it names the branch and prints the commands (ADR-032, `update.py`). Part 4, flow 4.7, draws the sequence.

Part 3. Data, identifiers and state

This part describes where each kind of record lives, how records refer to one another, and the state machine an attempt moves through.

3.1 The repository folders and who owns them

Walkthrough.

1. The top-left group is **program-owned**. The curriculum maintainer and the application developers change it upstream, and it arrives in a fork by merge.
2. The top-right group is **participant-owned**. It does not exist in a fresh clone: the first `start-quest` action creates `participant/progress.yaml`, `participant/ACTIVITY.md` and the first evidence package. After that the participant owns it, commits it, and may edit it.
3. The bottom group is **machine-owned** and ignored by Git. Deleting it loses nothing that a build cannot recreate, except the error page of the last failed build and the port files of running services.
4. The arrows show the only read and write paths. The application writes into `participant/` only through the store, the evidence code and the review code, and only inside the configured participant root (ADR-018). A read resolves symbolic links and re-checks the result; a write refuses one outright, one path component at a time (ADR-042, Part 2, Section 2.7).

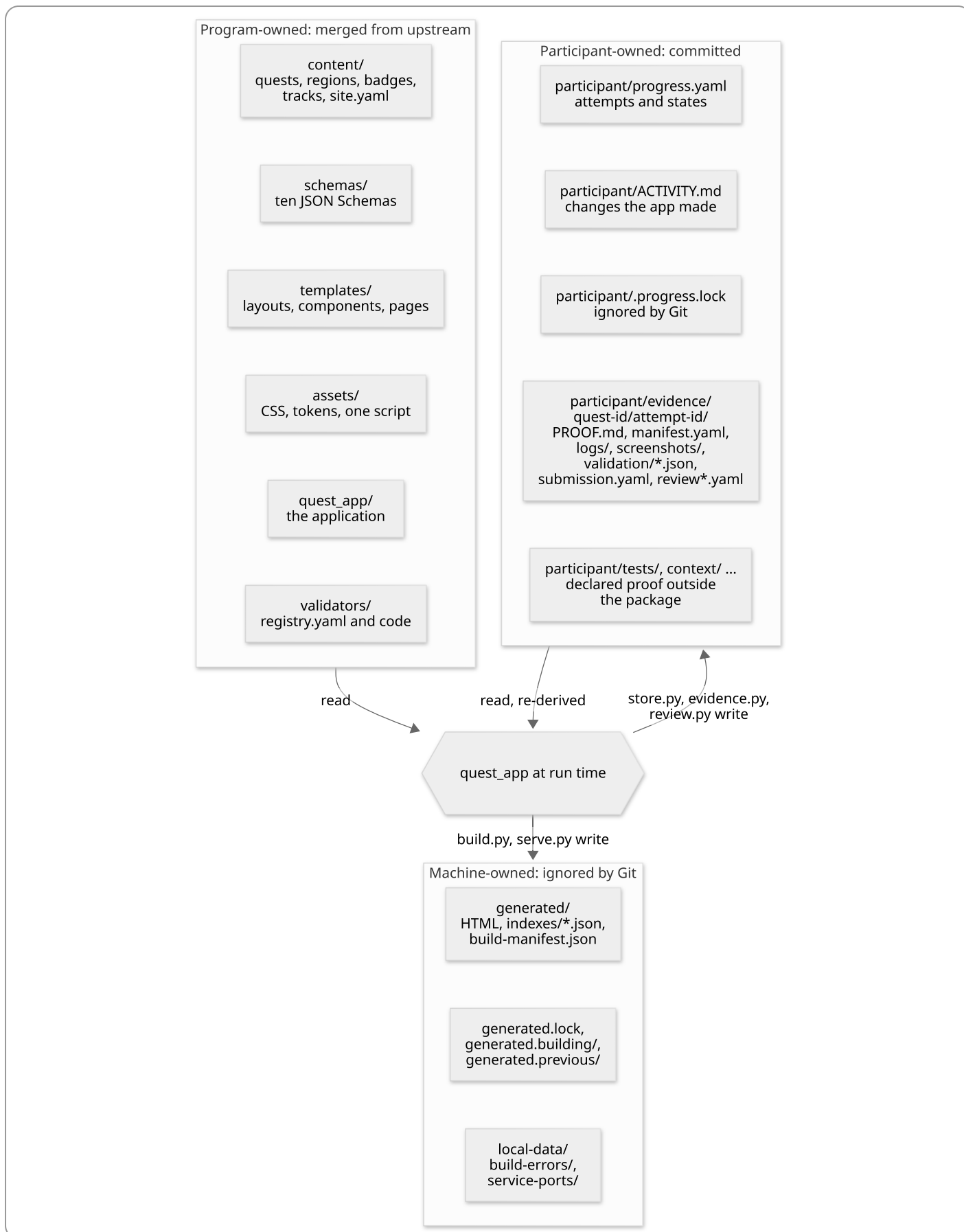


Figure 3. Repository folders by owner, from the code (`store.py` , `review.py` , `build.py` , `.gitignore`).

Where this differs from the plan. `docs/ARCHITECTURE.md` lists participant files the application does not create or read: `PROFILE.md` , `QUEST-PASSPORT.md` , `QUEST-LOG.md` , `skills/` , `scripts/` . The passport is a generated

page, not a participant file. The code creates only what the figure shows; a quest may ask the participant to create other folders, such as `participant/tests/`, as proof.

3.2 Records and what each one holds

Record	Format and location	Written by	Holds
Quest	Markdown with YAML front matter, <code>content/quests/<region>/<name>.md</code>	Maintainer	<code>id</code> , <code>version</code> , <code>title</code> , <code>summary</code> , <code>region</code> , <code>level</code> , <code>xp</code> , <code>estimated_minutes</code> , <code>tags</code> , <code>outcomes</code> , <code>proof</code> (required and optional), optional <code>prerequisites</code> , <code>validators</code> , <code>related_quests</code> , <code>risk</code> ; the body holds Mission, Scenario, Acceptance criteria, Required evidence, Safety constraints, Hints, Reflection
Region, badge, track, site	YAML under <code>content/</code>	Maintainer	Ordering and outcomes (region), criteria and authority (badge), quest selection (track), identity and navigation (site)
Validator registry	<code>validators/registry.yaml</code>	Program	Every program that may run, with its constraints (Section 2.6)
Progress	<code>participant/progress.yaml</code>	Application, and the participant by hand	Participant identity, selected track, focus tags, and one entry per attempt : <code>attempt_id</code> , <code>quest_id</code> , <code>quest_version</code> , <code>content_hash</code> , <code>state</code> , <code>timestamps</code> , <code>evidence_path</code> , <code>submission_id</code> , <code>review_id</code>
Evidence package	<code>participant/evidence/<quest-id>/<attempt-id>/</code>	Participant (content), application (template)	<code>PROOF.md</code> answering what was built, where, how to reproduce, what was checked, what remains, and a sensitive-values confirmation; <code>manifest.yaml</code> ; <code>logs/</code> , <code>screenshots/</code>
Validation result	JSON, <code>validation/<run-id>.json</code> inside the package	Validator runner	Run ID, validator ID and version, quest and attempt IDs, timestamps, outcome, checks, redacted output excerpt
Submission	<code>submission.yaml</code> inside the package	Application, on submit	Submission ID, quest version, attempt ID, content hash, evidence hash , proof_files digests, secret-scan result, validation run IDs, advisories
Review	<code>review.yaml</code> inside the package; superseded decisions archived as <code>review-<timestamp>-<6 hex>.yaml</code> (round 16 finding S2: the random suffix keeps two decisions recorded in the same second from overwriting each other's archive)	Application, on the reviewer's decision	Review ID, quest ID and version, attempt ID, evidence hash, reviewer display name, decision, findings, verification statement (for approval), <code>proof_files</code>
Activity log	<code>participant/ACTIVITY.md</code>	Application	One line per change the application made
Generated site	<code>generated/</code>	Build	HTML pages, JSON indexes, build manifest. Never authoritative

Every participant-side record is validated against its schema before it is written and again when it is read. Every one of them is also read bounded, and never through a symbolic link: a state file, a submission or a review record up to 2 MB, a validation result up to 8 MB (Part 5, Section 5.9). Round 12 found that only `progress.yaml`

had that bound; a FIFO or a link to a device in place of a validation result or a stray `review*.yaml` could hang or crash `build` (findings E2 and E3).

3.3 Stable identifiers

Records refer to each other by **stable ID**, never by filename or title (ADR-012). An ID is lowercase ASCII letters, digits and hyphens, starts with a letter, is unique within its type, and is never reused for a different concept. Examples: `base-camp-repository-safety` (quest), `jira-jungle` (region), `validate-jira-read-assigned` (validator).

IDs the application mints:

ID	Shape	Example
Attempt	<code><first word of quest id>-attempt-<nnn></code>	<code>base-attempt-001</code>
Submission	<code>submission-<UTC timestamp>-<6 hex></code>	<code>submission-20260924142343-ca18d5</code>
Review	<code>review-<UTC timestamp>-<6 hex></code>	<code>review-20260924142355-1a2b3c</code>
Acceptance criterion	<code>ac-<n></code> , by position in the list	<code>ac-3</code>

Why IDs and not paths. Titles and filenames change. A reviewer's finding on `ac-3`, an attempt's `quest_id`, a badge's list of quests all survive a rename. URLs are derived from IDs by `routes.py`; content never stores an output path. Records are also connected by ID and never by modification time: a validator judges the package of the attempt it was given, not the newest file under `participant/evidence` (ADR-039).

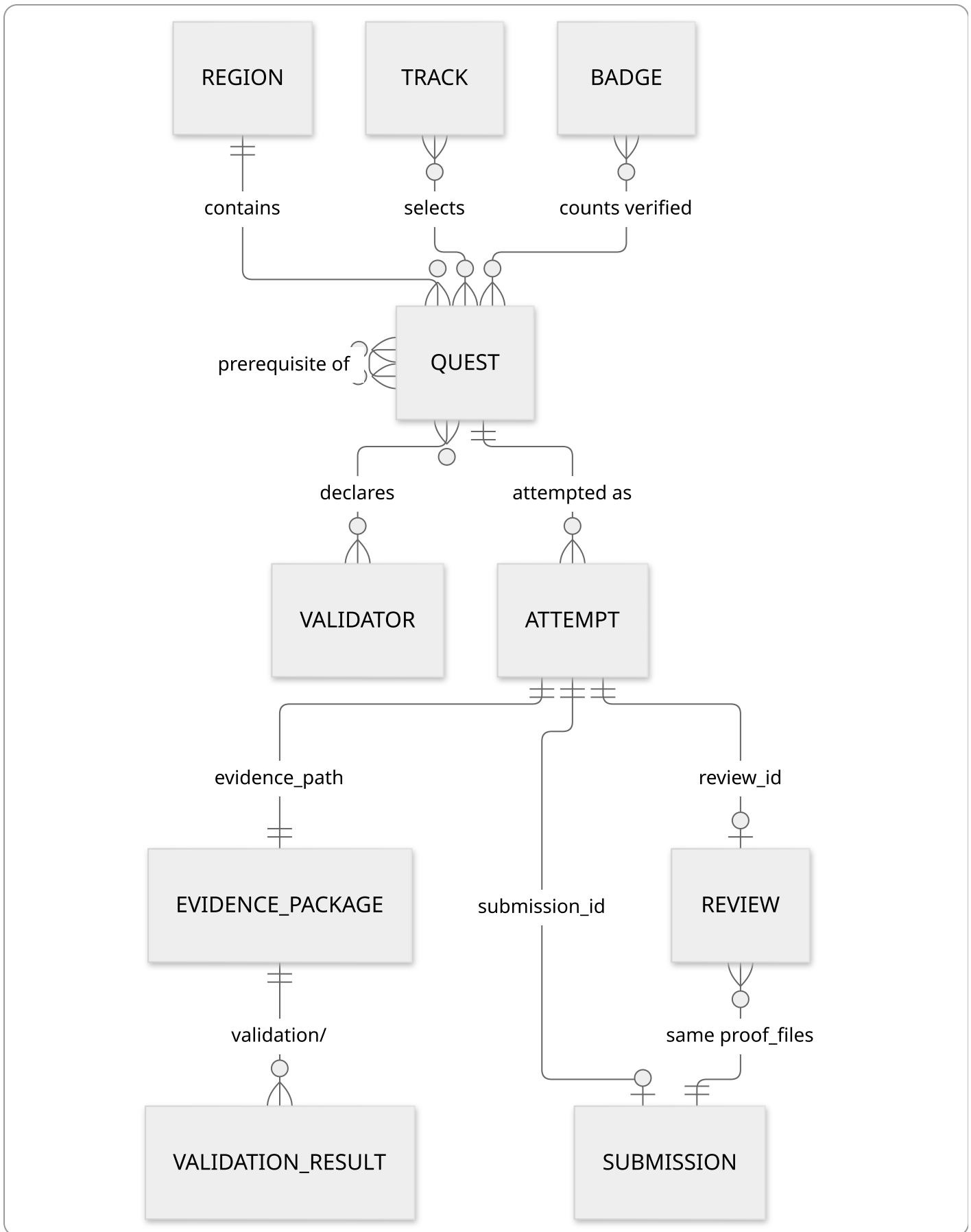


Figure 4. How records reference one another by stable ID. Program-owned records (region, track, badge, quest, validator) are on the left of each relation; participant-owned records (attempt and everything under it) on the right.

3.4 Versions and hashes

Three fingerprints let the application notice change without trusting anyone's word.

- **Quest version** (`version` , a positive integer) is declared by the author and must be raised when outcomes, criteria, proof, validators, XP, prerequisites or safety rules change. An attempt records the version it started on and keeps it through an update.
- **Content hash** (`content_hash`) is a SHA-256 over the quest's front matter and body. An attempt records it at start. A mismatch later is a **warning**, never an error: "the text changed since you started", or for a verified attempt "the approval may be stale" (ADR-028). It is a warning because the hash also changes on a typo fix, and a typo fix must not revoke someone's approved work.
- **Evidence hash** (`evidence_hash`) is a SHA-256 over the evidence package, excluding `validation/` , `submission.yaml` and the review records (ADR-031). It answers "has the participant's work changed?", so the application's own bookkeeping stays out of it. Hashing streams each file a megabyte at a time rather than holding it in memory, so a large log costs the build time, not memory (round 12 finding E8). "The review records" is now one definition, `progress.review_archive_paths` , shared by the loader, the evidence hash and `review.review_history` ; before round 12's finding E3 the loader checked `review-*.yaml` and the history read the looser `review*.yaml` , so a participant's own file such as `review-notes.yaml` was hashed by one and not the other, and a stray one with broken YAML passed `validate` and then crashed `build` . Because most quests declare proof outside the package, submissions and reviews also record **proof_files**: a digest per declared proof path outside the package, `missing` when nothing is there and `unresolvable` when the path leads outside `participant/` (ADR-031, amended in round 11).

Both hashes normalize a file's bytes and its name before hashing, not after (ADR-031, amended rounds 15, 16 and 17), so the digest answers "did the participant's work change?" rather than "did a checkout tool touch it?". A file that decodes as UTF-8 is hashed with every `\r\n` or lone `\r` folded to `\n` , so a Windows participant's checkout and a Linux or macOS reviewer's are not "changed" by Git's own line-ending conversion alone; a file name is NFC-normalized first, so the precomposed and decomposed Unicode forms of the same name (macOS commonly writes the latter) hash alike. Content that does not decode as UTF-8 — a binary file, or a UTF-16/32 export with no ASCII in it — is hashed exactly as it sits on disk instead: an earlier rule normalized any NUL-free file regardless of encoding, which folded a genuine byte of a UTF-16 character as though it were a line ending and hid a real edit behind an unchanged hash (round 16 finding E10). Content that is not UTF-8 is still folded, but only its `\r\n` pairs and only when Git's own `core.autocrlf` would convert it — no NUL byte, no lone `\r` , and Git's printable-byte ratio — so a cp1252 or Latin-1 log converted by Git is not a change either (round 17 finding E14). A directory symbolic link inside the package is hashed by its link text, as a file link is, so retargeting it is a change (round 17 finding E15).

The submission records both fingerprints. The reviewer's approval is refused if either has changed since submission unless the reviewer acknowledges the change, and after approval the loader warns if the package or any proof file changes.

3.5 The attempt state machine

Terms. A **quest state** is what the pages show for a quest. Two quest states are *computed* and never stored: **locked** (a prerequisite is not yet verified) and **available** (every prerequisite is verified, no attempt exists). The other six are **attempt states**, stored in `progress.yaml` : `in_progress` , `evidence_ready` , `locally_validated` , `submitted` , `needs_changes` , `verified` . A prerequisite counts as met only when it is *verified*, not merely attempted.

Walkthrough.

1. `start-quest` creates the attempt and its evidence package, and moves the quest from available to `in_progress` . It is refused on a locked quest, and it needs a confirmation.

2. `mark-evidence-ready` is the participant's assertion that the proof is assembled. It is refused if the secret scan finds anything in the evidence package.
3. `mark-locally-validated` is refused unless every validator the quest declares has a qualifying latest result. A quest with no validators reaches it on the participant's word. Running a validator is **not** a transition.
4. `submit-for-review` works from `evidence_ready` or `locally_validated`. Unrun or failing validators become advisories on the submission, not blockers. A secret-scan hit, a link leading outside the package, or a missing package blocks it. It needs a confirmation.
5. `reopen-evidence` and `withdraw-submission` let the participant step back to `in_progress`.
6. The reviewer's `record-review` moves `submitted` to `verified` (approved) or to `needs_changes` (needs changes, or rejected). It needs a confirmation. `resume-quest` takes the participant from `needs_changes` back to `in_progress`.
7. `verified` has no outgoing transition.

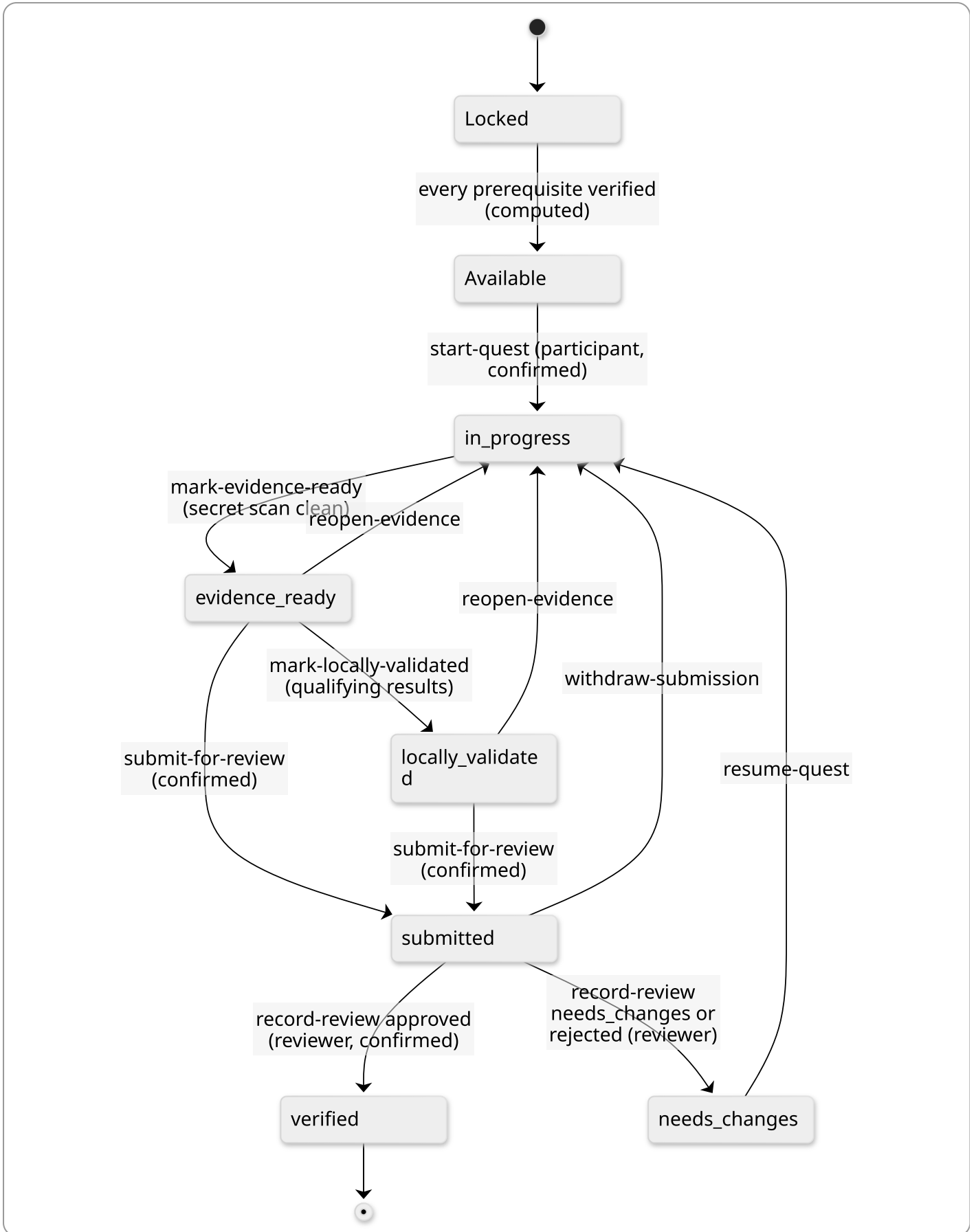


Figure 5. The attempt state machine as implemented in `quest_app/state_machine.py` (participant transitions) and `quest_app/review.py` (reviewer transitions). Locked and Available are computed in `progress_calc.py` and never written.

The rule that only a reviewer produces verified

Nothing in the transition table can produce `verified`. The one line of code that writes it is `review._apply_decision`, and it runs only after `record_decision` has checked that the attempt is `submitted`, that an approval carries a verification statement of at least twenty characters, and that the evidence has not changed since submission (or that the reviewer acknowledged the change).

Because `progress.yaml` is the participant's file, writing `state: verified` by hand is possible. The loader therefore re-derives `verified` on every load and refuses it, as the integrity error `progress.unverified_verified_state`, when the attempt names no review, the review is missing, belongs to another attempt or quest, is not an approval, has no verification statement, or approves a different quest version. The same applies to a hand-written `locally_validated` without a qualifying result for each declared validator (`progress.unvalidated_locally_validated_state`, ADR-017 amended in round 11), and to a hand-written `submitted` with no readable `submission.yaml` behind it (`progress.unsubmitted_submitted_state`, round 12 finding E4): `submitted` is the shadow of a request for review exactly as `verified` is the shadow of an approval, and without this check a reviewer could approve straight to `verified` without the record that submission is supposed to guarantee exists — including the secret scan `submit-for-review` runs before that record is ever written. `record-review` checks the same thing again for itself, so a `submitted` state built some other way than the loader cannot reach a decision either.

`progress.unvalidated_locally_validated_state` has one exception, added by round 12's amendment to ADR-017: when the attempt's `quest_version` is not the quest's current version, an upstream update may have *added* a validator since the attempt validated, and the application keeps no record of what an earlier version required. A missing validator is then a warning, `progress.locally_validated_missing_new_validator`, naming the validator and inviting a re-run — but only when the attempt has at least one qualifying result already; an attempt with none at all, version mismatch or not, is exactly the forgery this check exists to catch, and stays an error.

Verified XP and badges are computed only from attempts that pass this check. Claimed XP counts `evidence_ready` and every later state.

Where the code and the specification disagree

`docs/ARCHITECTURE.md` draws a different machine. The code is authoritative; each difference is deliberate unless marked open.

Specification	Code	Reason
<code>EvidenceReady --> InProgress: Validation fails</code>	No such edge. A failing run leaves the state alone	ADR-017: a validator is not an authority over participant state
Submission only from <code>LocallyValidated</code>	Submission also from <code>evidence_ready</code> , with unrun or failing validators carried as advisories	<code>review.readiness_problems</code> ; the specification's own note allows submission with advisory failures if displayed
No step back from evidence-ready or submitted	<code>reopen-evidence</code> and <code>withdraw-submission</code>	Participant can correct work before a reviewer decides
Decision <code>rejected</code> (<code>CONTENT-MODEL.md</code>) with no state of its own	<code>rejected</code> maps to the <code>needs_changes</code> state	<code>review._apply_decision</code>
Spelling <code>needs-changes</code> (<code>CONTENT-MODEL.md</code>)	<code>needs_changes</code> everywhere	<code>models.Decision</code> , <code>models.AttemptState</code>

Round 12 found two places the code fell short of its own design, both since fixed: a hand-edited `submitted` with no `submission.yaml` loaded and could be approved, skipping the secret-scan gate (finding E4, closed by `progress.unsubmitted_submitted_state` above), and a legitimate `locally_validated` attempt became a load error the moment an update added a validator to its quest (finding E5, closed by the ADR-017 amendment above). Neither is a disagreement any longer.

Part 4. The system in motion

This part walks through the main flows. Each diagram is drawn from the code, and each flow names the functions that implement it so a reader can check.

4.1 How to read these diagrams

The activity diagrams are UML-style, drawn as Mermaid flowcharts:

- a **lane** (a labelled box) groups the steps one role or one process performs;
- a rounded box is an action; a diamond is a decision; a circle marks the start; a double circle marks the end;
- a box whose text begins **Refused** is a place the flow stops with a message and changes nothing.

Every mutating action in these flows passes through `ActionRunner.perform ( quest_app/actions.py )`, whichever surface asked. It performs three steps before the action itself: it refuses `start-quest`, `submit-for-review` and `record-review` without a confirmation; it takes the progress lock; and it loads and validates the whole world (`pipeline.load_world`). If content or participant state does not validate, every action is refused. The diagrams show these steps once, in flow 4.2, and abbreviate them after that.

The commands in this part are the command-line equivalent of the browser controls. Run them from the repository root with the virtual environment active (`source .venv/bin/activate`).

4.2 A participant starts a quest and assembles evidence

Walkthrough.

1. The participant chooses an available quest and confirms "Start this quest and create an evidence package in my repository".
2. The action layer checks the confirmation, takes the progress lock, and loads the world. It refuses a quest ID that is not in the loaded content, so no caller-supplied string ever reaches the filesystem as a path.
3. It refuses a locked quest: every prerequisite must be *verified* (ADR-033). This check is in the action layer, not only in the page, so the CLI cannot bypass it.
4. `store.start_attempt` creates `progress.yaml` if this is the participant's first action, refuses if the quest already has an attempt, mints the attempt ID, and creates the evidence package from a template: `PROOF.md` with its six headings, `manifest.yaml`, and empty `validation/`, `screenshots/` and `logs/` folders. Existing files are never overwritten.
5. It appends the attempt (quest version and content hash included), validates the new `progress.yaml` against its schema, writes it atomically, and appends a line to `ACTIVITY.md`.
6. The site is rebuilt. A failed rebuild becomes an advisory; the attempt is still recorded (ADR-038).
7. The participant does the work in their own editor and tools: artifacts under `participant/`, and `PROOF.md` filled in.
8. `mark-evidence-ready` runs the secret scan over the whole package. Any finding, any file the scan cannot read, and any symbolic link leading outside the package blocks it. The message names the file and line, never the value.

BASH

```
quest-app action start-quest --quest base-camp-repository-safety --confirm
quest-app action mark-evidence-ready --quest base-camp-repository-safety
```

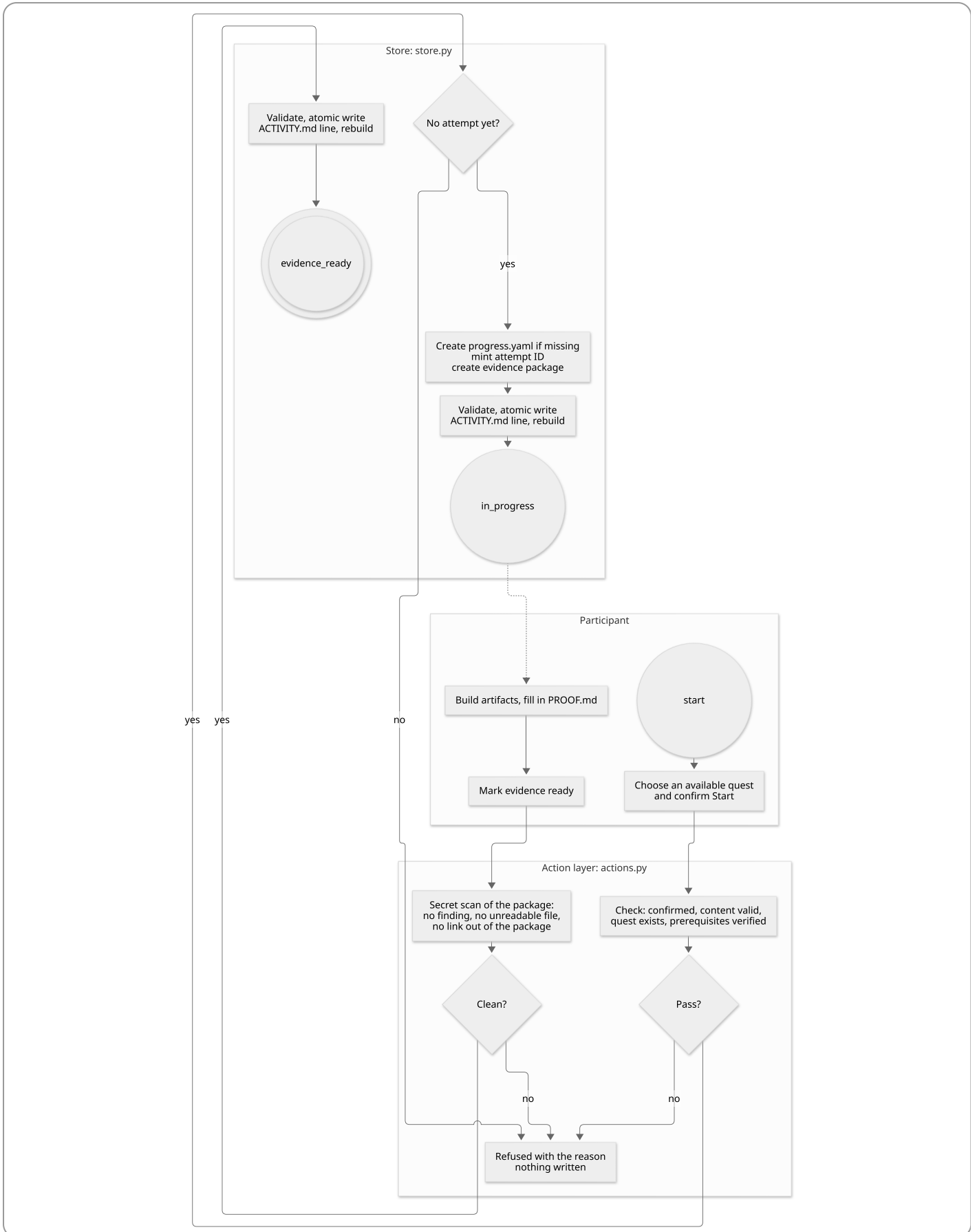


Figure 6. Starting a quest and marking evidence ready (`ActionRunner._perform`, `store.start_attempt`, `ActionRunner._require_clean_secret_scan`). Marking evidence ready needs no confirmation, but it takes the lock and loads the world like every action.

4.3 Running a validator

Walkthrough.

1. The participant runs one of the quest's validators. The action layer refuses a validator the quest does not declare, and a quest with no attempt.
2. The registry entry is looked up by ID. `run_validator` refuses a validator not registered for this quest, and binds parameters against their declared types and allowed values; there is deliberately no free-text parameter type.
3. The runner builds the child's environment from a minimal `PATH` plus the registry allowlist, sets the working directory from the registry, and starts `python -s -c <bootstrap>` in a new session and process group. The specification (roots, parameters, the attempt's evidence root) is sent on standard input. There is no shell and no argument built from input.
4. The child imports the entrypoint, but only from the `validators` package, and calls it with a `Workspace` and an output collector. The validator's own prints are moved to standard error so they cannot corrupt the result, which is written as JSON on the original standard output.
5. The parent reads both streams as they arrive. Five things can end the run: the child exits; the timeout passes; a stream exceeds 1 MiB; the child exits and one second passes while grandchildren still hold a pipe; or the child's result channel (its `stdout`) closes while the OS process is still alive a second later, which means something the validator left running — a non-daemon thread is the case this exists for — is holding the interpreter open after it already returned its result (round 12 finding E11). In every case the whole process group is killed.
6. Each reported check is checked against the result schema's own `id` pattern and `outcome` enum before it is trusted. One outside either is replaced with a check reporting that a check was malformed, and the run's outcome is forced to `environment_failure`: a validator that cannot describe its own check correctly has a defect in itself, not the participant's work (round 12 finding E6). The run is then classified (next table). Every result carries at least one check, so an `interrupted` or `environment_failure` run still tells the participant what happened.
7. The complete output and every check field are redacted *first*, then truncated: the excerpt to the registered limit (at most 20,000 characters), each check field to its own schema limit. Redacting before truncating, never after, means a secret-shaped token cannot be cut in half at the boundary and stored in clear (round 12 finding E7).
8. The result is validated against `validation-result.schema.json` and written atomically to `validation/<run-id>.json` in the attempt's package. The site is rebuilt. **The attempt's state does not change.**
9. Later, `mark-locally-validated` succeeds only if every declared validator's latest result is `pass` or `warning`.

Outcome	How it is reached (<code>validator_runner.py</code>)	Qualifies?
<code>interrupted</code>	The timeout passed	No
<code>environment_failure</code>	Output over 1 MiB; nonzero exit that the validator's own process produced; empty output; unreadable JSON; a check outside the schema's <code>id</code> or <code>outcome</code> ; the validator raised; the validator tried to leave its roots; or it reported an environment failure itself	No
<code>inconclusive</code>	No checks, every check skipped, or any check inconclusive (and none failed)	No
<code>fail</code>	Any check failed	No
<code>warning</code>	Any check warned, none failed or were inconclusive	Yes
<code>pass</code>	Every check passed	Yes

A nonzero exit is trusted over anything printed, *except* when the runner produced it itself by killing a process whose result channel had already closed (step 5's fifth case): that exit status describes the runner's own cleanup, not a judgment the validator passed on itself, so the result it already wrote is kept rather than discarded (round 12 finding E11).

BASH

```
quest-app action run-validator --quest base-camp-repository-safety \  
  --validator validate-repository-foundation  
quest-app action mark-locally-validated --quest base-camp-repository-safety
```


channel with a killed leftover process no longer read as `interrupted` (E11). **Still open, as a known limitation, not a defect (E9, Low):** a grandchild that starts its own session survives cleanup, because nothing short of an operating-system sandbox reaches a process that has left the group `killpg` targets (`docs/VALIDATOR-CONTRACT.md`; deferred item D9).

4.4 Submitting for review

Walkthrough.

1. The participant confirms "Submit this evidence for review. A reviewer will read it".
2. `review.readiness_problems` collects every problem at once, so the participant sees all of them together. **Blocking** problems: the attempt is not `evidence_ready` or `locally_validated`; the package is missing; or the scan found a secret, a link leading outside the package, or a file too large for the scan to read — three kinds of scan finding, worded separately by `evidence.scan_kinds` so a participant is not sent looking for a credential that was never there (round 12 finding C3). **Advisory** problems: a declared validator has not been run, or its latest result does not qualify.
3. Any blocking problem refuses the submission.
4. Otherwise `create_submission` computes the evidence hash and the proof-file digests, writes `submission.yaml` (validated against its schema) into the package, moves the attempt to `submitted`, records the submission ID on the attempt, and appends an activity line.
5. The site is rebuilt, and the result carries the advisories and the Git commands for the next step.
6. **The application never pushes, opens a pull request or merges** (ADR-032). It prints the commands: create a branch, add the evidence and `progress.yaml`, commit, push, open a pull request. The participant runs them. Pushing is a claim, on the participant's behalf, that the work is ready, and that claim is theirs to make.

BASH

```
quest-app action submit-for-review --quest base-camp-repository-safety --  
confirm
```

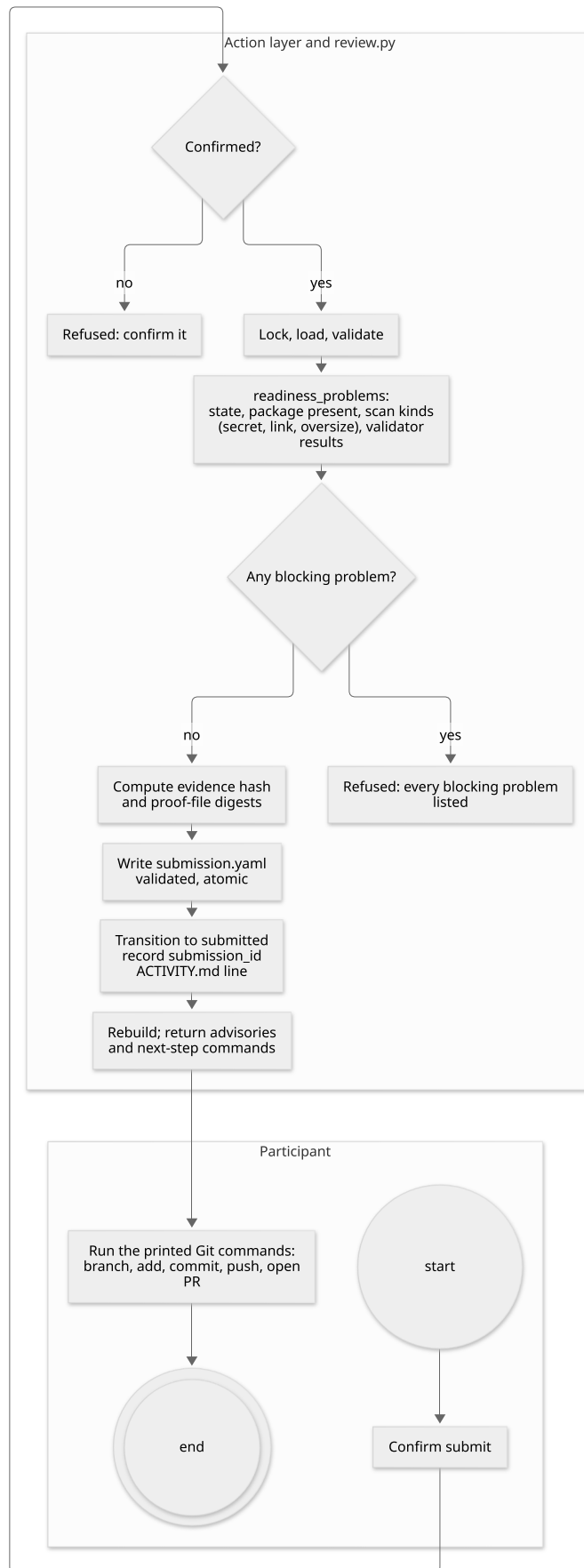


Figure 8. Submitting for review (`ActionRunner._submit`, `review.create_submission`, `review.readiness_problems`, `review.submission_instructions`).

4.5 The reviewer's decision

Walkthrough.

1. The reviewer reads the evidence: the reviewer page (`/review/<quest-id>/`) shows the quest and its version, the attempt, the submission, the validator results, the secret-scan result, and whether anything changed since submission, naming each changed path.
2. The reviewer chooses **approved**, **needs changes** or **rejected**, fills in the form, and confirms. The confirmation text depends on the decision; only the approval text says it produces verified XP.
3. `record_decision` refuses a decision that is not one of the three values, an attempt that is not `submitted`, and — since round 12 finding E4 — an attempt with no readable `submission.yaml`. The loader already refuses a hand-edited `submitted` state with no submission record at load (Part 3, Section 3.5); this check is `record_decision`'s own, so a `submitted` state built or altered some other way cannot reach a decision either.
4. An **approval** needs a verification statement of at least twenty characters saying what was checked and how. Then the **changed-since-submission gate**: the current evidence hash and each recorded proof-file digest are compared with `submission.yaml`. If anything differs, the approval is refused unless the reviewer acknowledges the change. Approving changed evidence would approve something the reviewer has not seen.
5. **Needs changes** and **rejected** need at least one finding with a severity, a summary and the evidence observed.
6. The review record is validated against its schema. An existing `review.yaml` is archived as `review-<timestamp>-<6 hex>.yaml` rather than overwritten, so a change of mind stays visible. The new record is written atomically.
7. `_apply_decision` sets the attempt state: `verified` for an approval, `needs_changes` otherwise. This is the only code that writes `verified`.
8. The site is rebuilt. On the next load, `verified` is re-derived from this record (Part 3, Section 3.5), and verified XP and badges follow from it.
9. After `needs_changes`, the participant runs `resume-quest` and the cycle repeats.

BASH

```
quest-app action record-review --quest base-camp-repository-safety \  
  --decision approved \  
  --reviewer "Your Name" \  
  --statement "What you checked and how, in your own words." \  
  --confirm  
  
quest-app action record-review --quest base-camp-repository-safety \  
  --decision needs_changes \  
  --reviewer "Your Name" \  
  --finding "high:What is wrong:Where you saw it" \  
  --confirm
```

Add `--acknowledge-changed-evidence` to approve evidence that changed after submission; the browser form has the same checkbox.

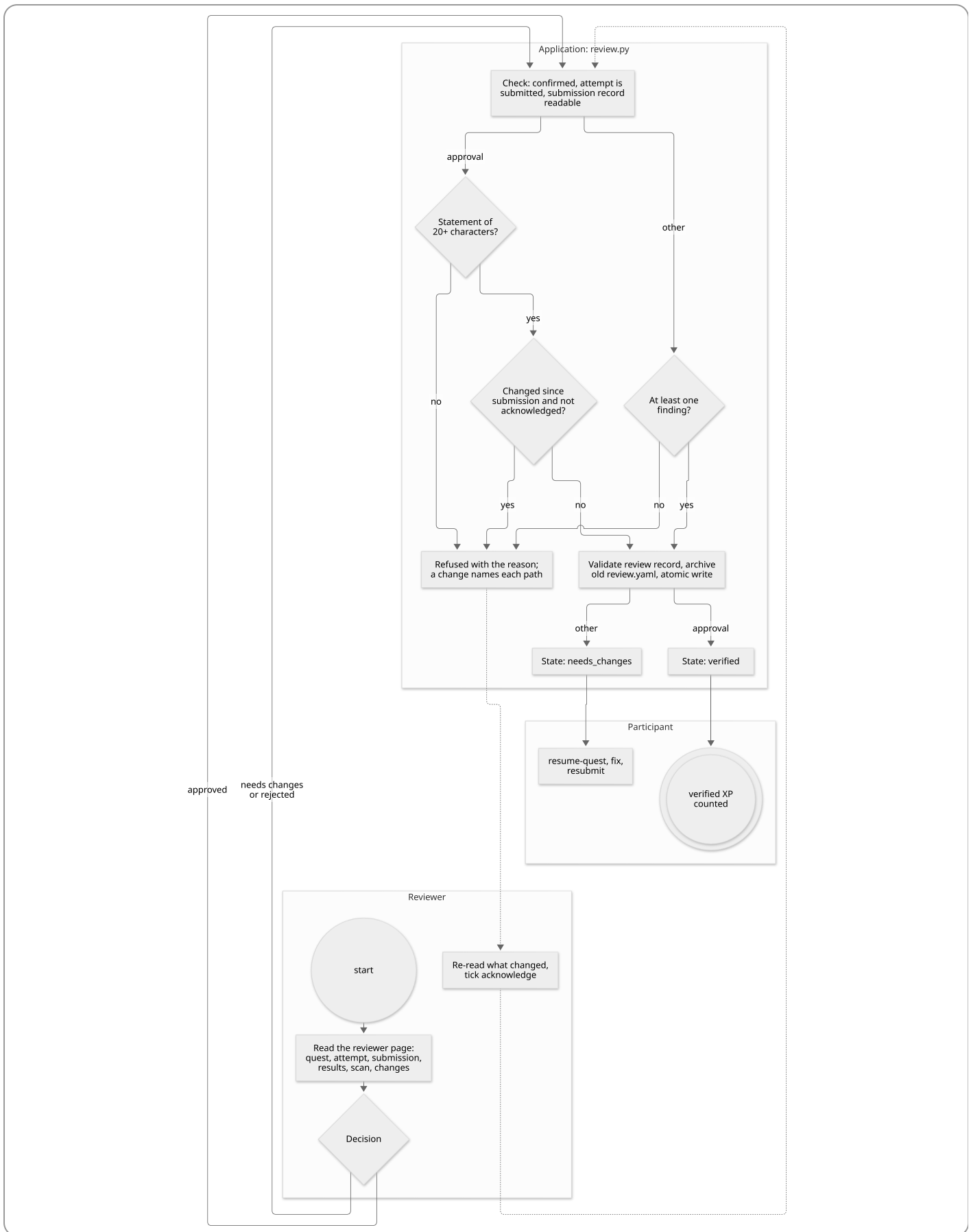


Figure 9. Recording a reviewer decision (`ActionRunner._record_review`, `review.record_decision`, `review.changes_since_submission`, `review._apply_decision`).

Provenance. A reviewer is identified by the display name in the record and by Git history. Records are not signed (ADR-030). The application refuses every internally inconsistent claim, but a participant with write access to their

own repository could write a record naming someone else. Programs that need more should review through pull requests so Git carries the identity. Release one also has no role separation: a participant can open the reviewer page for their own work (deferred item D11).

4.6 The build and render pipeline

Walkthrough.

1. `quest-app build` (also `make build`), `quest-app serve`, and every action's rebuild all call the same pipeline.
2. `load_world` loads content first: discover, parse strictly, validate against schemas, normalize, then apply cross-document rules and check validator references. Participant state is read only if content is valid, because an attempt cannot be judged against a curriculum that does not make sense. A missing `progress.yaml` is normal.
3. Participant state is read with bounded reads: each file must be an ordinary file under a size ceiling, 2 MB for `progress.yaml`, a submission or a review record, 8 MB for a validation result. Round 12 found this bound applied only to `progress.yaml`: a FIFO or a link to a device in place of a validation result, or a stray `review*.yaml` with broken YAML, could hang or crash `build` (findings E2 and E3). Both record kinds are now read the same bounded way, and review records are enumerated by one shared definition (`progress.review_archive_paths`) instead of two globs that used to disagree. A `validation/` directory that is itself a link is reported and nothing in it is read. `verified` and `locally_validated` claims are checked against their records.
4. **If anything is an error**, the build renders the error page (U11) to `local-data/build-errors/index.html`, prints its path, exits non-zero, and leaves the published site untouched.
5. **If valid**, `build_site` takes the build lock and renders into `generated.building`: derived states, totals, badges and recommendations; one view model per page; every page through Jinja2 with autoescaping; the JSON indexes and the manifest; a copy of `assets/`.
6. The publish is two renames. Only then does a reader see the new site.

BASH

```
make validate-content
make build
```

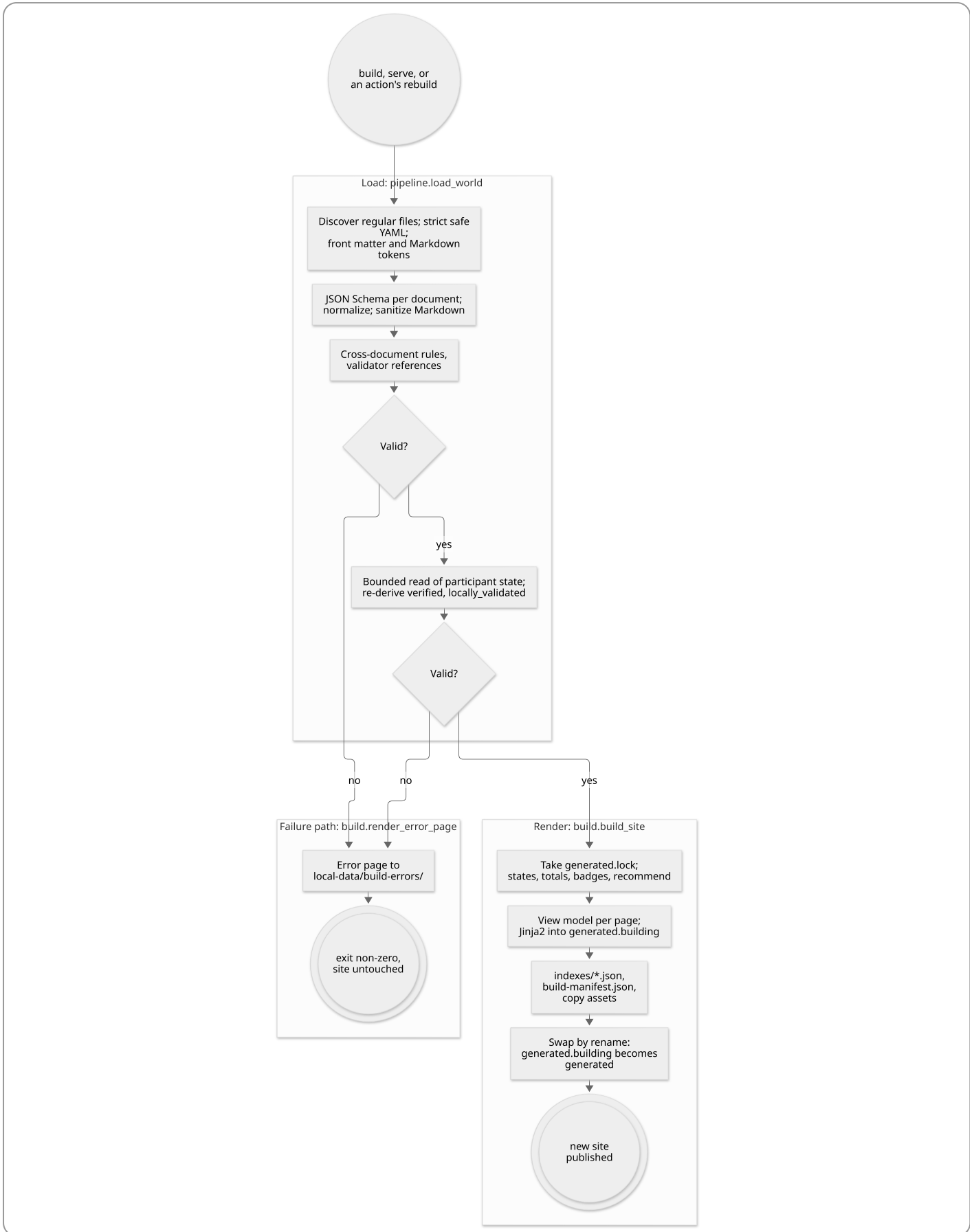


Figure 10. The build pipeline (`pipeline.load_world` , `content_loader.load_content` , `build.build_site` , `build._swap` , `build.render_error_page`).

Fixed since round 12. Findings E2 and E3 (bounded reads for validation results and review records, above) and E8 are closed: evidence files are hashed by streaming a megabyte at a time instead of holding each one in

memory, and the secret scan reads at most 2 MB of a file before treating it as a scan finding rather than pattern-matching a file that used to take a build about a minute and hold both locks while it ran.

4.7 Taking an upstream curriculum update

Walkthrough.

1. The participant runs the preflight. It checks: a Git repository; no merge in progress (checked first, because the working-tree advice would otherwise be to commit conflict markers); a clean working tree; an `upstream` remote; the participant folder. Each is `pass`, `warning` or `fail`.
2. It reports what a migration would do and which attempts are on an older quest version than upstream now publishes. Those attempts are not moved.
3. If anything fails, it prints the fix for each and exits non-zero. Otherwise it prints a backup branch name and the commands, and stops. It runs no merge and creates no branch.
4. The participant runs the printed commands: create the backup branch, fetch, read the incoming log, merge. Because curriculum and participant work are in separate folders, the merge touches only program-owned files unless both sides edited the same file.
5. The participant runs `make validate-content`. If it reports that the progress schema is old, they run `make migrate`.
6. The migration refuses during a merge, takes the progress lock, reads the file with a bounded read, applies each step (never advancing an in-progress attempt's quest version, never dropping an unknown field), validates, writes atomically, reloads the whole world, restores the original bytes if the reload fails, and — since round 12 finding E10 — appends an activity line naming the migrations it kept, the same as every other writer of `progress.yaml`.
7. If the merge goes wrong: `git merge --abort`, or reset to the backup branch.

BASH

```
quest-app update
make update-check
make migrate
```

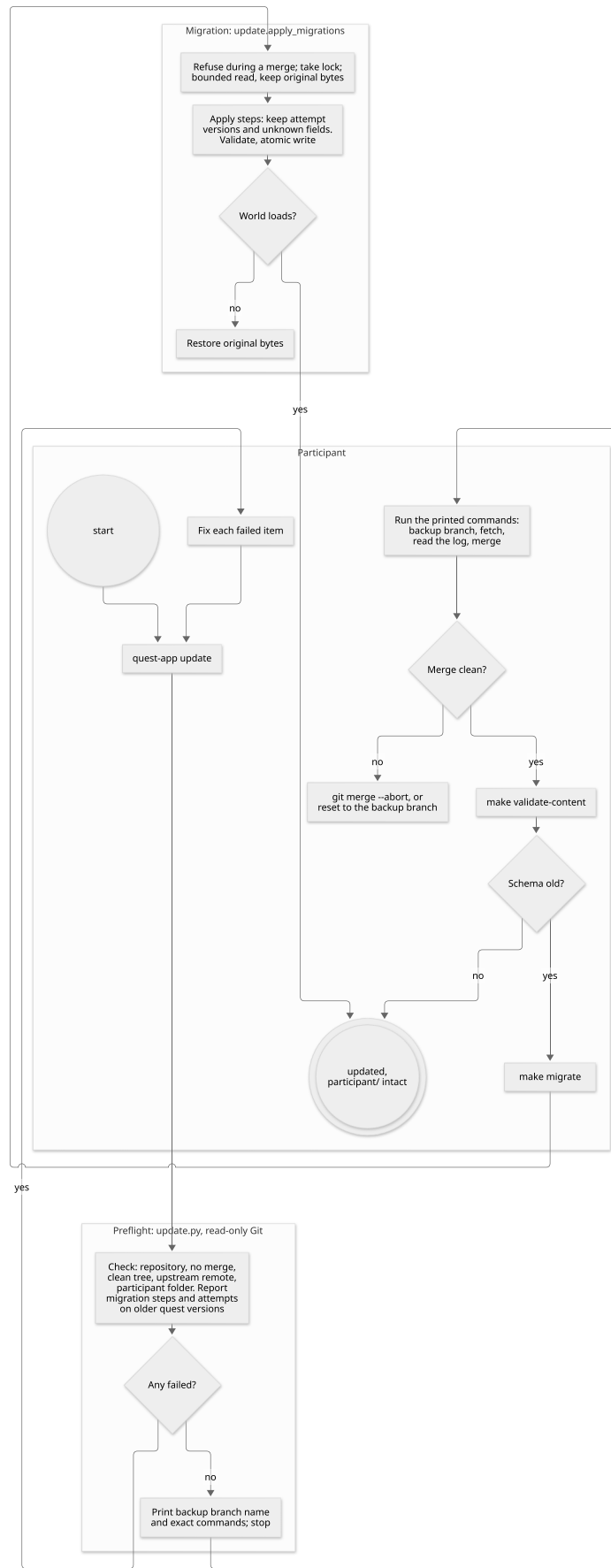


Figure 11. Taking an upstream update (`update.preflight`, `update.update_instructions`, `update.apply_migrations`, `migrations.migrate`). The flow starts at `start` in the Participant lane.

Fixed since round 12. A `locally_validated` attempt used to become a load error the moment an update added a validator to its quest, with every action then refused, including the `run-validator` that would have

cleared it (finding E5); ADR-017's amendment (Part 3, Section 3.5) now warns instead when the attempt still has a qualifying result from before the update. `make migrate` now writes the activity line above (finding E10).

4.8 One browser action through the loopback service

Walkthrough. The participant presses a button on a served page, for example Submit on the quest page. The page is an ordinary HTML form; no JavaScript is needed.

1. The browser posts `application/x-www-form-urlencoded` to `/api/action/submit-for-review/<quest-id>`. The form carries the per-run token, which the service substituted into the page when it served it, and the confirmation checkbox.
2. **Host check.** A `Host` header that is not a loopback name is refused, which defeats DNS rebinding (ADR-041).
3. **Origin check.** If `Origin` or `Referer` is present, it must be `http` on a loopback name at the bound port.
4. **Body.** The content type must be the form type and the declared length at most 64 KiB, checked before the body is read.
5. **Token.** Compared in constant time with the token in memory.
6. **Allowlist.** The action must be in `MUTATING_ACTIONS`; the path carries only IDs.
7. **Confirmation.** A confirmed action without the checkbox is sent back to the page with the confirmation text. `ActionRunner.perform` checks the same rule again, so the JSON endpoint and the CLI meet it too (ADR-033).
8. **Locks.** The service lock serializes requests in this process; the progress lock serializes against a CLI action in another process.
9. **Load, guards, write.** The action layer loads the world, applies the action's guards, and writes records atomically.
10. **Rebuild.** The site is rebuilt with live controls, because the service is running.
11. The service answers `303 See Other` back to the page, with a flash message for a refusal or an advisory. Every failure path answers with a status and a message; none escapes as a traceback.

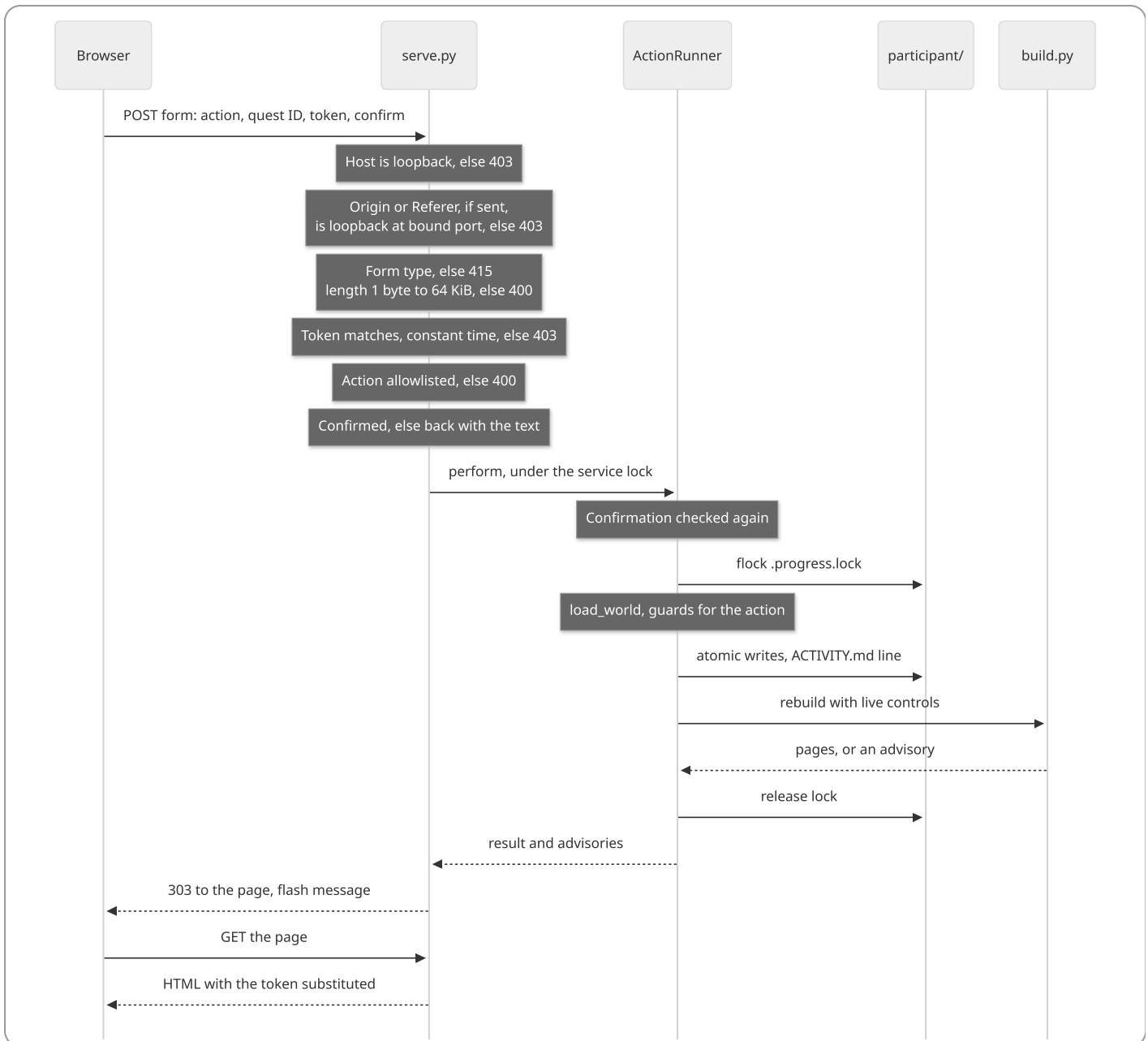


Figure 12. A form action through the loopback service (`serve.Handler._post`, `_handle_form_action`, `_host_is_acceptable`, `_origin_is_acceptable`, `_read_form_body`, `_token_matches`, `ActionRunner.perform`).

Fixed since round 12. Finding C1 (Blocking) was that every page was served with `Referrer-Policy: no-referrer` (both the header and the `<meta name="referrer">` tag), so Chromium sent `Origin: null` on a same-origin form POST, and `_origin_is_acceptable` at step 3 parsed `null` as a non-`http` origin and refused it — only the CLI, which sends no `Origin` at all, could change state, and no test had submitted a form end to end. The policy is now `same-origin`: a same-origin POST carries its real origin and passes the check exactly as a CLI request does; a cross-origin POST still sends no `Origin` and is still refused, unchanged. Verified against real Chromium, not assumed. Two browser tests submit real forms (start-quest, record-review), accept the confirmation, and check the resulting files on disk; both fail with 403 if the old policy is put back.

Part 5. The security model

A local application still has attackers. Every program on the machine can reach a loopback port, every web page the participant opens can try to post to it, and the repository it reads can arrive from someone else as a pull request. This part states the threats and the control that answers each, with the code that implements it.

5.1 Threats

From `docs/SECURITY-AND-PRIVACY.md`, restated as what the application must prevent:

1. A web page in the participant's browser making the local service change state (cross-site request forgery, DNS rebinding).
2. Any caller running an arbitrary command or reading or writing an arbitrary path through the service or the CLI.
3. Malicious Markdown in curriculum or evidence running script in the generated pages.
4. A validator reading credentials, escaping its folders, hanging, or flooding output.
5. A secret reaching evidence, a log, a validation result or Git.
6. A forged `verified` state, or an approval of evidence the reviewer never saw.
7. A hostile repository (a participant's fork checked out by a reviewer) crashing or hanging the application through symbolic links, FIFOs, huge files or deeply nested YAML.

5.2 The loopback boundary

The service is the only component that accepts requests, so it carries most of the controls. A request meets them in this order (`quest_app/serve.py`):

Control	Rule	Why
Loopback bind	<code>assert_loopback</code> refuses any bind address other than <code>127.0.0.1</code> , <code>::1</code> or <code>localhost</code> at startup	Nothing off the machine can connect
Host check	A request whose <code>Host</code> is not a loopback name gets 403 before anything is rendered (ADR-041)	A page at an attacker's domain that resolves to <code>127.0.0.1</code> is same-origin to the browser and could read a served page and take the token from it
No body on GET	A GET with a declared body is refused	An undeclared body on a kept-alive connection would be parsed as a second request with attacker-chosen headers
Origin check	For state-changing requests, <code>Origin</code> and <code>Referer</code> , when present, must be <code>http</code> on a loopback name at the bound port	A page on another site cannot post here. A missing header is accepted because a command-line client sends none; the token is what authorizes
Strict body	JSON or form content type only, declared length at most 64 KiB, checked before reading	Bounds memory and parsing
Per-run token	Minted at startup, held in memory, substituted into served HTML, compared in constant time; never written to disk or printed	Only a page this run served can act. A page opened from disk has only the placeholder

Control	Rule	Why
Action allowlist	The action must be in <code>MUTATING_ACTIONS</code> ; the quest ID must exist in loaded content; a validator must be declared by the quest	No path and no command crosses the boundary: the caller names IDs, the server resolves them
Confirmation	<code>start-quest</code> , <code>submit-for-review</code> and <code>record-review</code> need an explicit confirmation, checked in the action layer for every caller (ADR-033)	The actions that create files, hand work to someone else, or produce verified XP happen only when a person said so in that request
Response headers	Content Security Policy <code>default-src 'none'; script-src 'self'; style-src 'self'; img-src 'self' data:; form-action 'self'; base-uri 'none'; frame-ancestors 'none'</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy: same-origin</code> , <code>Cache-Control: no-store</code>	No remote script, style or frame; no framing by another page
Catch-all	Both handlers answer every exception with a status and the exception type, never a traceback	A traceback would print absolute paths and leave the client with no answer after a change had landed

Fixed since round 12 (finding C1, Blocking). The header, and the matching `<meta name="referrer">` tag on every page, used to be `no-referrer`, which strips the browser's `Origin` header to `null` on a same-origin form POST too. The origin check then refused every browser form as cross-origin; only the CLI, which sends no `Origin` at all, could change state, and no test had submitted a form in a real browser. `same-origin` still sends nothing off-site — a cross-origin POST is refused exactly as before — but a same-origin POST now carries its real origin and passes the check. Verified against real Chromium; two browser tests submit real forms and check the resulting files on disk.

5.3 No arbitrary command, no arbitrary path

- **Commands.** There is no code path from a request to a shell. Validators are Python callables imported by an allowlisted module path from the `validators` package, started in a child with a fixed argument list. Validator parameters are typed enumerations or numbers; there is no string parameter type. `git_status.py` runs exactly five read-only Git commands and raises if asked for another; `update.py` runs three read-only ones (ADR-014, ADR-032).
- **Paths.** Participant paths in records keep the contract prefix `participant/` and are resolved by `AppConfig.resolve_participant_path`, which rejects absolute paths, `..` segments and null bytes, resolves symbolic links, and re-checks that the result is inside the participant root (ADR-018). A validator's `Workspace` applies the same rule to its registered roots. The service resolves static file paths the same way under `generated/`. Writing uses a stricter, separate rule: `AppConfig.participant_write_path` keeps the *lexical* location rather than resolving links, because resolving a link and then checking the result is exactly what would hide a link a write should refuse (ADR-042, Section 5.4).
- **Deletion.** `tools/clean.py` (behind `make clean`) removes an exact allowlist of machine-owned paths and refuses anything reached through a symbolic link.
- **Participant root.** A configured participant root that is, is inside, or contains one of the application's own folders (`content`, `schemas`, `templates`, `assets`, `validators`, `quest_app`, `generated`, `local-data`) is refused before anything is written there (`_refuse_participant_root_inside_program_owned_folders`, ADR-042 amended round 15 finding E13): `GTQ_PARTICIPANT_ROOT=./content` used to land every participant write inside authored curriculum instead. The comparison folds case (round 16 finding S1): a plain `Path` comparison missed that `./Content` names the same directory `content/` does on a case-insensitive filesystem (macOS's default APFS, Windows), which reopened the same gap there.

5.4 Allowlisted writes

The application writes to these places only:

Target	Writer	Notes
<code>participant/progress.yaml</code>	<code>store.ProgressStore.write</code>	Schema-validated before writing, atomic, under the progress lock
<code>participant/ACTIVITY.md</code>	<code>store.append_audit</code>	One line per change
<code>participant/evidence/<quest>/<attempt>/</code>	<code>store</code> , <code>evidence.store_result</code> , <code>review</code>	Template files on start (never overwriting), validation results, <code>submission.yaml</code> , <code>review.yaml</code> and archives
<code>generated/</code> , <code>generated.lock</code> , <code>generated.building/</code> , <code>generated.previous/</code>	<code>build.py</code>	Disposable
<code>local-data/build-errors/</code> , <code>local-data/service-ports/</code>	<code>build.py</code> , <code>serve.py</code>	Disposable
A validator's registered write roots	The validator, through <code>Workspace.write_text</code>	None of the shipped validators declares a write root

Fixed since round 12 (finding E1, Blocking). Three writes used to miss this rule: the activity line, appended with a plain `open("a")` that followed a symbolic link out of `participant/` and hung forever on a FIFO named `ACTIVITY.md` while holding both locks; validation results, written into a `validation/` directory that could itself be a link leading outside the package; and the review archive, written with a plain `write_text`. Every participant write now goes through `safe_io.atomic_write` or `safe_io.append_to_regular_file` (ADR-042), which walk from the participant root one component at a time with `O_NOFOLLOW`, refuse a component that is a link or not a directory, refuse a target that exists and is not a regular file, and never open anything in a way that can block. When the refused target is `ACTIVITY.md`, the line is skipped and a warning goes to stderr, because the change it describes is already on disk by the time it runs; for every other target the write is refused before anything changes, like any other write failure.

Ceilings on what is read from the participant tree.

Ceiling	Applies to	Over it
2 MB (<code>MAX_STATE_BYTES</code>)	<code>progress.yaml</code> , <code>submission.yaml</code> , <code>review.yaml</code> and its archives	Reported by <code>validate</code> ; not read
8 MB (<code>MAX_VALIDATION_RESULT_BYTES</code>)	A validation result	Refused when written by <code>evidence.store_result</code> ; reported by <code>validate</code> if found anyway
2 MB (<code>MAX_EVIDENCE_FILE_BYTES</code>)	Each evidence file the secret scan reads (not images, PDF or zip)	A scan finding that blocks submission and tells the participant to trim the file (round 12 finding E8)

Evidence hashing streams a megabyte at a time rather than holding a file whole in memory, so a large file costs a build time, not memory.

5.5 Untrusted content in the pages

- Quest Markdown is rendered with raw HTML disabled, then sanitized by `nh3` with an explicit tag, attribute and URL-scheme allowlist (ADR-021).
- Jinja2 autoescapes every template value.
- `PROOF.md` is rendered through the same path. A symbolic link inside the package that resolves outside it is not rendered.
- Validator output is shown as text, never as HTML.
- The search index holds published quest text only, never evidence.
- Pages load no remote script, font, image or analytics, and the CSP forbids them.

5.6 Validator containment

Section 2.6 lists the controls: constructed environment, repository-first import path, own process group, timeout that kills the group, 1 MiB stream ceiling, output limit, redaction of output and of every check field before truncation, and validation of the result against its schema, including of each individual check (round 12 finding E6). `Workspace.write_text` carries the same write rule as the participant tree (Section 5.4): a special file at the resolved target, such as a FIFO, is refused rather than opened, which used to block a validator until its own timeout. They bound what a validator does through the `Workspace` and the process. They are **not** an operating-system sandbox: nothing blocks a socket, a direct `open()` bypasses the roots, and a process that starts its own session escapes the group. Container or seccomp isolation is deferred (D9). Validators are program-owned and reviewed like application code.

5.7 Secret scan and redaction

- **The detectors.** `quest_app/secret_patterns.py` holds one set of patterns (AWS, GitHub, GitLab and Trello tokens, Slack, Google, Anthropic, OpenAI, Atlassian, Stripe and npm tokens, JSON web tokens, PGP private-key blocks, Basic- and token-scheme `Authorization` headers, credentials in URLs, and keyword assignments such as a `password` or `secret` field with a value). Every scan and every redaction uses it. Past this document's round 12 baseline: the GitLab and Trello patterns and the Basic/token-scheme header (round 15 finding E2 — "bearer headers" alone missed a Jira script's Basic-auth header) were added, and a pagination-field exemption and a PGP block-marker fix landed the same round (finding E12). The JWT and `basic-auth-url` patterns now bound how far their quantifiers can scan (round 15 finding E5): unbounded, either could make one scan of an adversarial multi- megabyte file take minutes to hours, held under the store, generated and service locks the whole time. Round 17 widened the net again, in two passes merged together: a whole private key block is one match, so redaction removes its body (E1); a credential word may carry a name suffix (`SECRET_KEY`, `DB_PASSWORD_PROD`), unless the suffix says the value only describes one (`password_file`, `tokenizer` ; E2, A-E1); CLI flags, login commands, cookies, Slack webhooks, Azure connection keys, compound XML, YAML block headers and Markdown- or HTML-escaped tokens are read (E4, L10, A-E2, A-E7, A-E8, A-E11); and the exemptions for code, templates and placeholders now match only the shapes they name (E3, A-E3 to A-E6, F12). `docs/SECURITY-AND-PRIVACY.md` lists each rule and its trade.
- **Repository hygiene.** `tools/secret_scan.py` (`make secret-scan`, part of `make check`) scans every tracked text file and prints `path:line:column` and the pattern, never the value. A line ending in an allow pragma is skipped; this is how the scanner's own test fixtures live in the repository.
- **The evidence gate.** `evidence.scan_evidence` scans the whole evidence package before `mark-evidence-ready` and again before submission. It calls the detectors directly, so a participant cannot switch the check off with the pragma. A file it cannot read, a link leading outside the package, and a file over the 2 MB scan ceiling (Section 5.4) all count as findings: the scan cannot clear what it may not read. Every finding is one of four kinds — secret, link, oversize or unreadable (`evidence.scan_kinds`) — and the evidence and review pages word each one separately, so none of the other three is reported as "something secret-like" (round 12 finding C3; round 17 separated unreadable). Images, PDF and archives are skipped by their leading bytes,

which is why a screenshot is not scanned (a known limitation); a text file that only opens with an ASCII signature is scanned (E7). A proof path in the quest's own evidence area is looked for only inside the attempt's real package, the one the scan and the hash read (round 17 L1).

- **Redaction.** Validator output and every free-text check field pass through `redact_text` *before* truncation, never after: cutting first can sever a secret-shaped token at the boundary, and the half that survives matches no detector's pattern (round 12 finding E7, fixed). Error messages replace path-shaped text with `<path>`.

Why a gate and not a report. A false positive costs the participant a minute. A missed credential costs a rotation. The scan is a safety net, and the documents say so: widening the detectors is deferred until there is a corpus to measure false positives against (D10).

5.8 Review integrity

- `verified` is re-derived from the review record on every load; an attempt that claims it without a matching approval is refused as an integrity error (Part 3, Section 3.5).
- An approval needs a verification statement; a request for changes needs a finding, which may name the acceptance criterion it is about (round 17 L9).
- An approval re-runs the secret scan and is refused on any finding, so evidence edited after submission cannot be verified holding a token (round 17 E5).
- An approval of evidence whose hash or proof-file digests changed since submission is refused unless the reviewer acknowledges it; after approval, a later change raises a warning that the approval may be stale.
- Reviewer identity is a display name plus Git history, not a signature (ADR-030). The limitation is stated in the reviewer guide and the interface.
- A `submitted` attempt needs a readable `submission.yaml` behind it: the loader refuses a hand-edited `submitted` with none as the integrity error `progress.unsubmitted_submitted_state`, and `record_decision` refuses to act on one even if it somehow reached that state some other way. Before this (round 12 finding E4, fixed), such an attempt loaded clean and could be approved straight to `verified`, skipping the secret-scan gate that `submit-for-review` applies before it ever writes that file.

5.9 Hostile repository files

A reviewer runs the application against a participant's repository, so participant files are treated as hostile input:

- content discovery reads only regular files inside the content tree (a link to `/dev/zero` or a FIFO is refused);
- `progress.yaml`, `submission.yaml` and every review record are read through `safe_io.read_bounded_bytes`: an ordinary file under 2,000,000 bytes, opened `O_NONBLOCK` and checked with `fstat` on the open descriptor, not a `stat` on the path beforehand, so a path swapped for a FIFO between the check and the open cannot hang it either;
- a validation result is read the same bounded way, up to 8,000,000 bytes (`MAX_VALIDATION_RESULT_BYTES`), and a `validation/` directory that is itself a link is reported and nothing in it is read;
- review records are exactly the files one shared definition, `progress.review_archive_paths`, enumerates — `review.yaml` and its `review-<timestamp>-<6 hex>.yaml` archives (the random suffix is round 16 finding S2: a timestamp alone named the same archive file for two decisions recorded in the same second, and the second silently overwrote the first; an archive written before this fix, timestamp alone, is still recognized) — used by the loader, `review.review_history` and the evidence hash alike;
- YAML is parsed with `StrictSafeLoader`: safe constructors, duplicate keys refused, deep nesting reported (ADR-025, ADR-029);
- the service-port files are read with a 16-byte bound.

Fixed since round 12 (findings E2 and E3, High). Validation result files, and every `review*.yaml` beyond `review.yaml`, used to be read without any of the bounds, above: a FIFO hung `validate`, `build` and the

service, a link to `/dev/zero` was read until the process was killed, and a stray unparsable `reviewer-notes.yaml` — which the loader's own check missed, because it globbed `review-*.yaml` while `review.review_history` globbed the looser `review*.yaml` — passed `validate` and then crashed `build` with an absolute path in the traceback. Both record kinds now go through the same bounded reader `progress.yaml` always used, and review records are read from the one shared list above rather than two globs that disagreed.

5.10 External systems and privacy

Release one performs **no external write** and makes **no outbound request** of its own. Every quest declares `risk.external_write: false`, every validator is registered `network: denied`, and `tests/test_release_facts.py` fails the day a quest declares a write. The preview-and-confirm requirements in `docs/SECURITY-AND-PRIVACY.md` are the conditions a write-capable quest must meet before it is added, not a mechanism that runs today. No telemetry leaves the machine.

Part 6. Key decisions

`docs/DECISIONS.md` is the full log: forty-three architecture decision records (**ADRs**), appended rather than rewritten, several amended after an audit found the code and the decision disagreeing. This part groups them by the question each one answers and gives the reason in one or two sentences. Read the log for the evidence behind each.

6.1 Product shape

ADR	Decision	Why
001	The quest is its own repository	Participants fork, run and update it independently of any other program
002	Participants fork the canonical repository	A fork keeps upstream history, so curriculum updates arrive by merge
003	Three ownership zones in separate top-level folders	Updates cannot overwrite participant work; generated output stays disposable
010	A restrained field-guide visual direction	Motivation without looking childish in an enterprise setting
011	Only a reviewer decision produces verified progress	Automated checks establish facts, not the quality or meaning of the work
013	Repository evidence is primary; screenshots support	Files, tests and logs are reproducible and diffable

6.2 Content and contracts

ADR	Decision	Why
004	Markdown with YAML front matter for quests, YAML for structured content	Readable diffs and approachable authoring
005	JSON Schema is the public contract	Language-neutral, testable, editor-friendly
012	Stable IDs; attempts record quest version and content hash	Titles and filenames change; version-aware attempts survive curriculum edits
016, 026	Acceptance criteria are parsed from the body with positional IDs <code>ac-<n></code> ; a bullet list is a warning, an empty item or split list an error	One copy of the criteria; the failures that actually move an ID are the ones that stop the build
020	<code>jsonschema</code> against the published schemas, frozen dataclasses inside	No second rule set that can drift from the schema a maintainer reads
021	<code>markdown-it-py</code> with raw HTML off, then <code>nh3</code> sanitization	The guarantee does not depend on the renderer's escaping
025, 029	YAML through a strict safe loader: no custom constructors, no duplicate keys, bounded nesting	<code>xp: 10</code> then <code>xp: 90</code> must not mean one thing in review and another at build
027	Markdown structure from the token stream, never from lines	Line regexes turned code blocks and nested items into criteria

ADR	Decision	Why
028	A content-hash mismatch is a warning, never fatal	A typo fix must not revoke approved work, but the reader must be told

6.3 Generation and runtime

ADR	Decision	Why
006	Python and Jinja2 generate the site deterministically	Server-rendered HTML is inspectable and needs no frontend build
007	No frontend framework	A framework would duplicate state the Python model already holds
008	Static generation plus a loopback service for the few mutating actions	Static pages cannot write files; a full web app is unnecessary
009	Mermaid documents; HTML and CSS define the UI	Diagrams describe relationships, not layout or accessibility
015	Generated output is not canonical	A build can always recreate it
022	The service uses the standard library <code>http.server</code>	Every requirement is a restriction; a framework hides the surface they apply to
019	Python 3.10 or newer (amended from 3.12)	The sandbox a participant may be given ships 3.10
023, 024	pytest, Playwright, axe-core; ruff and <code>mypy --strict</code>	One toolchain locally and in CI
037	The service records each bound port under <code>local-data/service-ports/</code>	A CLI action must know whether a service is running to build live or offline pages
040	The build honours <code>SOURCE_DATE_EPOCH</code>	Makes "two builds are byte-identical" true on demand

6.4 Safety and integrity

ADR	Decision	Why
014	Only registered validators and named actions run	Arbitrary command execution on localhost is an avoidable boundary failure
017	A validation run never changes attempt state; <code>locally_validated</code> is requested and guarded	A validator must not be an authority over participant state. Amended round 12: a version mismatch with at least one qualifying result warns instead of erroring when an update added a validator, because the application cannot tell that apart from the validator never having existed
018	Participant paths keep the <code>participant/</code> prefix; the root is configuration	Tests use fixtures without touching live work; the contract does not move. Amended round 12: the generated and local-data roots are configuration the same way, so a test that shells out to the CLI cannot rebuild the repository's own <code>generated/</code>
030	Reviewer provenance is conventional, and said so	A half-built signature scheme invites trust it has not earned

ADR	Decision	Why
031	The evidence hash covers the work, not the records; proof files outside the package are digested too	The application's bookkeeping must not read as a change to the work
032	The application never pushes, opens a pull request or merges	Those are claims the participant makes, not the tool
033	Every rule the browser shows is enforced in the action layer	A second caller turned a greyed-out button into a hole
039	A validator judges the attempt it was given	Records connect by ID, never by modification time
041	The service answers only to a loopback name, and every request gets a response	Defeats DNS rebinding; no traceback, no silent drop
042	Every participant write walks the tree component by component with <code>O_NOFOLLOW</code> , and an unusable <code>ACTIVITY.md</code> skips its line	Following a link was the escape route; blocking forever while holding both locks was the outage
043	The build deletes only what it made, and nothing the application owns is written through a link	A <code>generated</code> path that was a link, or that overlapped the curriculum, turned a routine rebuild into deleting someone else's files

6.5 Concurrency and failure

ADR	Decision	Why
034	One mutation at a time across processes: a file lock around load, decide and write	The CLI and the service could interleave a read-modify-write
035	One build at a time per output directory	Two builds shared one staging directory and published partial sites
036	Not being able to lock is never a reason to refuse the work	The atomic rename keeps files whole; refusing to record work is worse
038	A failed rebuild after an action is an advisory, not a failed action	The participant's record cannot be regenerated; the site can

6.6 Patterns across the decisions

Three patterns recur and explain most of the code's shape:

- One place per rule.** Confirmations live in `state_machine.CONFIRMATIONS`, guards in `actions.py`, secret patterns in `secret_patterns.py`, schemas in `schemas/`. A second copy is a second thing to drift, and audits repeatedly found drift where a rule had two homes.
- Re-derive, never believe.** Anything a participant is not allowed to decide is computed from records on every load rather than read from their file.
- Degrade toward the participant's record.** When something non-essential fails (a lock, a rebuild, an activity line), the participant's work is kept and the failure is reported, never the other way round.

Part 7. Where we are, and the phase plan

This part is the project's plan. It says what the goal is, where the work stands, how every phase of the remaining work runs, and which phases remain. It replaces the audit-status part of versions 1.x.

The rule this part exists to enforce: scope changes only at a phase gate. Between gates the current phase's scope is frozen. A new idea, from anyone, goes to `docs/PARKING-LOT.md` and is decided at the next gate, not acted on.

7.1 The goal

Put the Golden Thread Quest in front of real participants and a reviewer, learn from what they do, and grow the curriculum and the application in the order that learning says is most valuable.

The first release is a **pilot**: a small number of participants, working locally, on the quests that exist today, with the known limitations written down (Section 7.6). It is not a finished curriculum and not a hardened product. Everything after the pilot is chosen at a gate.

7.2 Where we are (2026-09-29)

Current phase: Phase 2A, the pilot release. Gate 1 passed on 2026-09-29, when the program owner decided to release to a pilot. Phase 1's checklist passed and DH7 is closed (`docs/audits/phase-01-verify.md`). Phase 2A fixes the two parking-lot items the program owner chose and builds the release. Phase 2B is the pilot itself.

How we got here

The application is built. Stages 0 to 9 of `docs/IMPLEMENTATION-PLAN.md` were completed and audited on 2026-09-16 and 2026-09-17. Everything Parts 1 to 6 describe exists, with 1382 tests under `make check` and 57 browser-driven tests under `make test-ui`. Eight quests ship, one per region.

The last twelve days went to one open criterion. Stage 10, the release decision, has one unchecked criterion: DH7, "the final audit reports no unresolved blocking or high findings." Its closing rule required an independent round that finds no blocking or high finding at all. Seventeen rounds ran, from 2026-09-17 to 2026-09-29. Every round found at least one, every finding was fixed with a test, and the next round, pointed at new angles and at the new fixes, found more.

What that taught us. The rounds found real defects: round 12 found that every browser form was refused, and round 17 found that the page showed a proof path the gates did not read. But the closing rule had no bound. A fresh adversarial reviewer can always find something in 15,000 lines of code, so the loop could not end on its own, and each round widened the scope a little: a broader secret scanner, integrity checks, timing tests. None of those widenings was a decision the program owner made at a checkpoint, and no working build was shown to the program owner along the way.

What stops now. Round 18 is not run. The per-round audit loop is replaced by the phase method below. The audit records under `docs/audits/` stay as history.

Round	Blocking	High	Round	Blocking	High
1 (final audit)	3	4	10	1	8
2	1	3	11	0	3
3	1	2	12	2	4

Round	Blocking	High	Round	Blocking	High
4	0	10	13	2	3
5	0	9	14	0	3
6	2	6	15	0	3
7	1	8	16	0	2
8	0	7	17	1	5
9	0	8			

Each round's record is `docs/audits/round-NN-independent-audit.md`; rounds 1 to 3 are in `stage-10-final-audit.md` and `stage-10-final-re-audit.md`. An outside review on 2026-09-17 (`external-review-2026-09-17.md`) called the engine a release candidate and the curriculum too thin.

7.3 How every phase runs

Every phase has the same five steps. What changes from phase to phase is only the scope.

1. **Plan, then freeze.** At the gate before the phase, this part records the phase's goal, its scope as a numbered list, what is out of scope, a *done-when* checklist of things that can be shown true or false, the demo, and a budget in working sessions. Once the program owner approves it, the scope does not change until the next gate.
2. **Build.** Only the numbered scope items. Work happens on feature branches and merges to `main` on the implementer's judgment, as `CLAUDE.md` rule 2 allows. An idea that is not in scope goes to `docs/PARKING-LOT.md` with one line saying why it might matter.
3. **Verify.** One independent review, in a fresh context, checks the done-when list and nothing else. A blocking or high finding *inside the scope* is fixed, and only that fix is re-checked. A finding outside the scope goes to the parking lot. **At most two verify cycles;** if the phase still fails, it stops and goes to the gate as it is.
4. **Demo.** The program owner runs the phase's result, following written steps, and sees it work.
5. **Gate.** The design document gets a new minor version (the rule is in `REGENERATING.md`), recording what was done, what was learned and what changed. The parking lot is triaged: each item is scheduled into a phase, left in the lot, or cut. Then the program owner chooses the next phase and approves its plan.

Two stop rules end a phase early and go straight to the gate:

- the phase has used half again its budget (for example, a third session on a two-session phase);
- a safety or data-loss problem turns up outside the phase's scope. It is reported, not fixed on the spot, so the program owner decides whether it outranks the plan.

7.4 The phases

Every piece of remaining work the project knows about is in one of these phases or in the parking lot. Phase 1 is planned in full. Later phases are sketched, and each one is planned in full at the gate before it, using what the earlier phases taught. Each item carries a recommendation: **do**, **decide** (at that phase's gate, likely on pilot evidence), or **cut** (recommended not to do in the foreseeable future).

Phase 0: Reset and plan (this version)

- **Goal:** stop the audit loop and put the plan in one document the program owner can read.

- **Done when:** this part is merged; `docs/PARKING-LOT.md` exists; `CLAUDE.md` states the phase rule; `PLANNING-STATUS.md` points here.
- **Gate 0 (2026-09-29): passed.** The program owner approved Phase 1 as written and the closing-rule change in its item 2.

Phase 1: Pilot release decision (done)

- **Goal:** decide on a fixed checklist whether the application as built goes to a pilot, and let the program owner see it working.
- **Budget:** two sessions.
- **Scope:**
 1. **Housekeeping.** Remove the stale round 17 worktrees under `.worktrees/`, their local branches and `backup/round-17-before-rewrite`. Delete the remote `chore/round-*` and `fix/r*` branches that are merged into `main`.
 2. **Replace DH7's closing rule** with the release checklist in item 3: DH7 closes when every item in it passes and one verify pass finds no blocking or high finding *within those flows*. Approved at Gate 0.
 3. **The release checklist**, run for real, not by test client:
 1. CI is green on `main`, every job.
 2. A true `git clone` from GitHub, then `make setup`, `make check`, `make setup-ui` and `make test-ui`, all pass.
 3. **Participant flow:** open the site with `make serve`, pick a quest, start it, build its evidence, run its validator, and submit it.
 4. **Reviewer flow:** open the submission, ask for changes with a finding; the participant fixes and resubmits; the reviewer approves; the passport shows verified completion and verified XP, while claimed progress stays visibly separate.
 5. **Maintainer flow:** add a sample quest as one Markdown file; after `make build` it appears in its region, the catalog, search and its tag page with no change to UI code.
 6. **Failure flow:** break a content file; the build names the file and field, and the last good site stays up.
 4. **A demo walkthrough**, `docs/guides/DEMO.md`: the checklist's flows 3 to 6 as numbered steps the program owner can follow in about fifteen minutes.
 5. **The release record.** `docs/audits/final-audit.md` gets the checklist results and a recommendation, expected to be `release-with-advisories`; Stage 10's boxes in `docs/IMPLEMENTATION-PLAN.md` are checked; `docs/RELEASE-NOTES.md` and the `README` status note say "pilot".
- **Out of scope:** round 18's targets; every item in Phases 3 to 7; any new hardening.
- **Done when:** items 1 to 5 are complete, every checklist item passes, and the verify pass finds no blocking or high finding within the checklist's flows.
- **Demo:** the program owner follows `docs/guides/DEMO.md`.
- **Result:** every checklist item passed. CI on `main` first failed on a start-up race, which pull request 18 fixed. The verify pass found three Low wording problems in the demo, all fixed. Five observations are in the parking lot. See `docs/audits/phase-01-verify.md`.
- **Gate 1 (2026-09-29): passed.** The program owner decided to release to a pilot of about two participants, after fixing the two parking-lot items about missing proof.

Phase 2A: Pilot release (current)

- **Goal:** a tagged release that two participants can take up from a Slack message, with the two missing-proof gaps closed.
- **Budget:** one session.
- **Scope:**

1. **Refuse a submission that is missing required proof.** A required file, folder or command record that the evidence page shows as *Not detected* or *Needs attention* (empty) blocks submission, and the refusal names each one. Validators stay advisory at submission, as they are today, and demonstrations cannot be detected, so they are not checked.
 2. **An unfilled `PROOF.md` fails the Base Camp check.** A proof document that is still the template, with headings and nothing under them, is a `fail`, not an advisory. The check still looks at what the document says, not whether files exist, as `docs/VALIDATOR-CONTRACT.md` asks.
 3. **The release.** Version 0.2.0, the README and release notes say "pilot", and a Git tag `v0.2.0` has a GitHub release whose notes link the pilot guide.
 4. `docs/guides/PILOT.md`, for participants. It covers what they need installed, forking and setup, the two pilot quests (Base Camp required, BA Ruins' transcript quest if there is time), how to hand in evidence, how to send feedback, and a warning that forks of a public repository are public, so the pilot uses fictional material only.
 5. `docs/guides/PILOT-LEAD.md`, for the program owner. It holds the Slack message ready to paste, how to review a participant's evidence from their fork, what feedback to collect, and when the pilot ends.
- **Out of scope:** the other three parking-lot items and everything in Phases 3 to 7.
 - **Done when:** items 1 to 5 are complete, with tests for 1 and 2 that fail without the fix. `make check` and CI pass on `main`. A verify pass acting as a new participant follows `PILOT.md` from the release through a Base Camp submission, then follows `PILOT-LEAD.md`'s review steps, and finds no blocking or high finding.
 - **Demo:** the program owner reads the Slack message and `PILOT.md`, then sends the message.

Phase 2B: Pilot run

- **Goal:** watch two participants and one reviewer use the application, and collect what confuses, blocks or delights them.
- **Budget:** calendar time. The program owner sets the end date in the Slack message, and two weeks is suggested.
- **Scope:** code changes only for a defect that stops a participant from continuing. Every other observation goes to the parking lot.
- **Output:** a pilot report that ranks what to change. It is the main input to Gate 2 and to choosing among Phases 3 to 7.

Phase 3: Participant and reviewer experience

Known items, likely joined by pilot findings. The ones marked **do** were found while using the application; the others wait for evidence.

Item	Source	Recommendation
A degraded view for an attempt whose evidence folder is missing, instead of an unbuildable site	D12	Do: one deleted folder breaks everything today
Reviewer-awarded badges: a badge-award record so they can be granted	Known limitation 4	Do if the pilot uses badges
Role separation between participant and reviewer pages	D11	Decide
Success notices after an action	D14	Decide
Showing the quest version an attempt started on, not only the current one	Known limitation 10	Decide
Quest-page section order	D15	Decide

Item	Source	Recommendation
Deeper gamification than the quest framing implies	External review 9	Decide, with the curriculum

Phase 4 onward: Curriculum

The outside review's main finding was that the curriculum is thin: eight quests, one per region.

`docs/CURRICULUM-BACKLOG.md` proposes 128: Base Camp 9, Jira Jungle 23, Trello Islands 17, GitHub Caverns 18, Context Library 12, BA Ruins 13, Scrum Village 9, Playwright Labyrinth 20, and 7 hidden challenges.

- **One region per phase**, in an order chosen at each gate. The recommended first is **Base Camp**, because every other region builds on its safety quests.
- Each curriculum phase includes the validators its quests need and a glossary content type if the region's quests need one (D1).
- A quest that writes to Jira, Trello or GitHub must meet the preview-and-confirm conditions in `docs/SECURITY-AND-PRIVACY.md` first. The first region with such quests carries that capability as its own scope item.
- **Recommendation:** decide how many regions to build after the pilot. The full 128 is a plan, not a promise.

Phase 5: Engineering hygiene

Item	Source	Recommendation
A scheduled CI job that runs a true <code>git clone</code> and setup	Known limitation 9	Do; Phase 1 runs it once by hand
Coverage reporting	D2	Decide
A fixture that holds all eight quest states at once	D8	Decide
A per-test timeout for the scaling tests	D16	Decide
Splitting the largest modules, with complexity limits	External review 7	Decide, only if a later phase is slowed by them

Phase 6: Security hardening

Release one's controls are documented in Part 5. These go further. They matter when validators or quests come from authors the program does not review, which is not the case today.

Item	Source	Recommendation
Container or seccomp isolation for validators	D9	Cut until third-party validators exist
Reaching a validator grandchild that starts its own session	Round 12 E9	Cut, same reason; it depends on D9
Wider secret-scanner coverage	D10	Decide, only on a real false negative from the pilot
Scanning binary and compressed files	Known limitation 8	Cut; the participant's confirmation in <code>PROOF.md</code> covers it
Type-tagged evidence hash chunks	D13	Cut; it re-flags every approved attempt and closes a gap that hides nothing

Phase 7: New capabilities

Named in the specification and not built. Each is a product decision, not a fix.

Item	Source	Recommendation
Live Environment Health checks through the local service	D7	Decide after the pilot
A central, opt-in scoreboard	Product brief	Decide after the pilot
AI-assisted advisory evaluation of evidence	Product brief	Decide after the pilot
External-write previews	Product brief	Build with the first curriculum region that needs them

7.5 Where things are recorded

What	Where
The plan: goal, current phase, phase sketches	This part
Ideas that arrived mid-phase	<code>docs/PARKING-LOT.md</code>
What each version of the plan changed and learned	<code>docs/design/CHANGELOG.md</code>
The original stages and the deferred-work register (D1 to D16)	<code>docs/IMPLEMENTATION-PLAN.md</code>
Release criteria	<code>docs/ACCEPTANCE-CRITERIA.md</code>
Verify passes and audits	<code>docs/audits/</code>

7.6 Known limitations of release one

Deliberate boundaries, stated for participants in `docs/RELEASE-NOTES.md`. The phase that would lift each one is in brackets.

1. Recording a reviewer decision needs the local service or the CLI, because it writes files.
2. Reviewer identity is conventional, not cryptographic (ADR-030). [Phase 3, D11]
3. Environment Health shows build-time facts, not live ones. [Phase 7, D7]
4. Reviewer-awarded badges cannot yet be awarded; they show as pending. [Phase 3]
5. Validator isolation is policy plus process boundaries. [Phase 6, D9]
6. Catalog filtering needs JavaScript; region and tag pages work without it.
7. No coverage reporting, no glossary content type. [Phases 5 and 4, D2 and D1]
8. Binary and compressed files are not scanned for secrets. [Phase 6]
9. Clean-clone installation is tested in CI from a `git archive` export, not a real clone. [Phase 1 once by hand, Phase 5 on a schedule]
10. Earlier quest versions are not kept; pages show the current version's criteria. [Phase 3]
11. A validator grandchild that starts its own session outlives the timeout. [Phase 6]
12. Two builds are byte-identical only when `SOURCE_DATE_EPOCH` is set.

Part 8. Glossary

Terms are defined where they first appear; this list collects them. Code names are given where the term maps to one.

Term	Meaning
Acceptance criterion	One item of the ordered list under a quest's <code>## Acceptance criteria</code> heading. Identified by position as <code>ac-<n></code> with a hash of its text (ADR-016, ADR-026)
Action	One of the ten named operations that change state: <code>start-quest</code> , <code>mark-evidence-ready</code> , <code>mark-locally-validated</code> , <code>reopen-evidence</code> , <code>submit-for-review</code> , <code>withdraw-submission</code> , <code>resume-quest</code> , <code>run-validator</code> , <code>record-review</code> , <code>rebuild</code> (<code>actions.MUTATING_ACTIONS</code>)
Action layer	<code>quest_app/actions.py</code> , specifically <code>ActionRunner</code> : the one place every action's rules are enforced, called by both the service and the CLI (ADR-033)
Activity log	<code>participant/ACTIVITY.md</code> , one line per change the application made to participant files
ADR	Architecture decision record, numbered in <code>docs/DECISIONS.md</code>
Advisory	A problem reported alongside a successful action rather than blocking it, for example an unrun validator at submission, or a rebuild that failed after a change was recorded
AI Context Engineer	The role the curriculum trains: owner of the intent and validation bookends of software delivery
Atomic write	Writing a temporary file in the same directory, flushing it to disk, and renaming it over the target, so the target is either wholly old or wholly new (<code>store.atomic_write_text</code>)
Attempt	One participant's run at one quest, recorded in <code>progress.yaml</code> with an attempt ID, quest version, content hash, state and evidence path
Attempt state	One of the six stored states: <code>in_progress</code> , <code>evidence_ready</code> , <code>locally_validated</code> , <code>submitted</code> , <code>needs_changes</code> , <code>verified</code> (<code>models.AttemptState</code>)
Available	Computed quest state: every prerequisite verified, no attempt yet. Never stored
Badge	A recognition defined in <code>content/badges/</code> , awarded automatically from verified quests, or by a reviewer or the program (reviewer awards are not implemented in release one)
Blocking / High / Medium / Low	Audit finding severities. Blocking: unsafe, data-loss risk, architecture violation, or core flow unusable. High: material requirement failure or likely confusion. Medium: quality, resilience or accessibility weakness. Low: polish
Bookend	One of the two ends of delivery the role owns: intent (requirements, work items) and validation (tests, evidence)
Build lock	Exclusive file lock on <code>generated.lock</code> held for a whole build (ADR-035)
Claimed XP	XP from quests in <code>evidence_ready</code> or any later state. The participant's own record, never added to verified XP
Confirmation	An explicit "I mean this" in the request, required for <code>start-quest</code> , <code>submit-for-review</code> and <code>record-review</code> . A checkbox in the browser, <code>--confirm</code> on the CLI, <code>"confirm": true</code> in JSON (<code>state_machine.CONFIRMATIONS</code>)
Content hash	SHA-256 over a quest's front matter and body. Recorded on an attempt at start; a later mismatch is a warning (ADR-028)

Term	Meaning
Content problem	The single error shape every layer reports: file, stable ID, field path, rule, redacted received value, suggestion (<code>errors.ContentProblem</code>)
Curriculum maintainer	The person who writes quests, regions, badges, tracks and validators. Also called curriculum author
DH7	The one open release acceptance criterion: the final audit reports no unresolved blocking or high findings. Closes when Phase 1's release checklist passes and its verify pass finds no blocking or high finding within the checklist's flows (Part 7, Section 7.4; rule set at Gate 0)
Evidence hash	SHA-256 over an evidence package, excluding <code>validation/</code> , <code>submission.yaml</code> and <code>review*.yaml</code> (ADR-031)
Evidence package	The folder <code>participant/evidence/<quest-id>/<attempt-id>/</code> holding <code>PROOF.md</code> , <code>manifest.yaml</code> , logs, screenshots, validation results, the submission and review records
Fork	The participant's copy of the canonical repository; its <code>origin</code> remote. Curriculum arrives from the <code>upstream</code> remote
Generated output	Everything under <code>generated/</code> : HTML, JSON indexes, build manifest. Disposable and never authoritative (ADR-015)
Gate	The checkpoint between two phases: the design document is versioned, the parking lot triaged and the next phase approved. The only point at which scope changes (Part 7, Section 7.3)
Lens	One of the independent reviewers in an audit round, each with its own scope and worktree
Loopback service	The HTTP server in <code>serve.py</code> , bound to <code>127.0.0.1</code> , that serves the pages and performs actions
Locked	Computed quest state: at least one prerequisite is not verified. Never stored
Machine-owned	The ownership zone of <code>generated/</code> and <code>local-data/</code> : disposable, ignored by Git
Migration	A step that moves <code>progress.yaml</code> from one schema version to the next, validated before and after, reversible by restoring the original bytes (<code>migrations.py</code>)
Needs changes	Attempt state after a reviewer asks for changes or rejects. The participant resumes from it
Origin check	Refusal of a state-changing request whose <code>Origin</code> or <code>Referer</code> is present and is not <code>http</code> on a loopback name at the bound port
Parking lot	<code>docs/PARKING-LOT.md</code> : ideas and out-of-scope findings that arrived mid-phase, decided at the next gate
Participant	The learner who works through quests in their own fork
Participant-owned	The ownership zone of <code>participant/</code> : the participant's files, never replaced by an update
Phase	A unit of planned work with frozen scope, a done-when checklist, a demo and a budget, ending at a gate (Part 7, Section 7.3)
Preflight	<code>quest-app update</code> : read-only checks before taking an upstream update. Prints commands; runs no merge
Prerequisite	A quest that must be verified before another can start
Program-owned	The ownership zone of <code>content/</code> , <code>schemas/</code> , <code>templates/</code> , <code>assets/</code> , <code>quest_app/</code> , <code>validators/</code> and most of <code>docs/</code> : changed upstream

Term	Meaning
Progress lock	Exclusive file lock on <code>participant/.progress.lock</code> held across load, decision and write of every action (ADR-034)
Progress store	<code>store.ProgressStore</code> , the only writer of <code>participant/progress.yaml</code>
PROOF.md	The participant's narrative for one attempt: what was built, where, how to reproduce, what was checked, what remains, and a sensitive-values confirmation
Proof files	Digests of every declared proof path outside the evidence package, recorded on submission and review so a change to them is detected (ADR-031, amended)
Proof requirement	An entry in a quest's <code>proof.required</code> or <code>proof.optional</code> : a file, directory, validator result, command record, screenshot, demonstration or review
Qualifying result	A validator run whose outcome is <code>pass</code> or <code>warning</code> . Only these count toward <code>locally_validated</code>
Quest	One unit of curriculum: a Markdown file with YAML front matter under <code>content/quests/<region>/</code>
Quest version	An integer the author raises when a quest's evaluable meaning changes. An attempt keeps the version it started on
Redaction	Replacing secret-like values in validator output, check fields and messages before they are stored or shown (<code>secret_patterns.redact_text</code>)
Region	A group of quests on the map, such as Base Camp or Jira Jungle. Defined in <code>content/regions/</code>
Registry	<code>validators/registry.yaml</code> : the complete list of programs the application may run, with their constraints
Request token	A random value minted when the service starts, held in memory, substituted into served pages, and required on every state-changing request
Review record	<code>review.yaml</code> in the evidence package, or one of its <code>review-<timestamp>-<6 hex>.yaml</code> archives: the reviewer's decision, findings, verification statement, evidence hash and proof files. Enumerated by one shared definition, <code>progress.review_archive_paths</code> , used by the loader, the review history and the evidence hash alike (round 12, amended)
Reviewer	The person who evaluates submitted evidence and records a decision. Also called Questmaster
Round	One release audit pass against a specific commit, recorded under <code>docs/audits/</code> . Seventeen ran; the phase method replaced them (Part 7, Section 7.2)
Secret scan	Detection of secret-like values: over tracked files by <code>tools/secret_scan.py</code> , and over an evidence package by <code>evidence.scan_evidence</code> , where it blocks marking evidence ready and submission. A finding is one of three kinds — secret, link or oversize (<code>evidence.scan_kinds</code>)
Session guard	A session-scoped test fixture (<code>tests/conftest.py</code>) that fails the whole test run if the repository's own <code>generated/</code> , <code>local-data/</code> or <code>participant/</code> changed, catching a test that forgot to isolate one of the three writable roots (round 12)
Stable ID	A lowercase, hyphenated identifier that never changes when a title or filename does, and is never reused (ADR-012)
Submission record	<code>submission.yaml</code> in the evidence package: what was submitted, with its evidence hash and proof files. A <code>submitted</code> attempt with none readable is a load error, <code>progress.unsubmitted_submitted_state</code> (round 12)
Track	An ordered or rule-based selection of quests, defined in <code>content/tracks/</code>

Term	Meaning
Upstream	The canonical repository participants take curriculum updates from
Validator	A program-owned Python function, registered by ID, that checks participant artifacts and returns structured findings. Never an authority over state (ADR-017)
Validator outcome	One of <code>pass</code> , <code>fail</code> , <code>warning</code> , <code>environment_failure</code> , <code>inconclusive</code> , <code>interrupted</code>
Verification statement	The reviewer's own words, at least twenty characters, saying what they checked and how. Required to approve
Verified	Attempt state produced only by a reviewer's approval, and re-derived from the review record on every load (ADR-011)
Verified XP	XP from verified quests only
Verify pass	A phase's one independent review, which checks that phase's done-when list and nothing else (Part 7, Section 7.3)
View model	A display-ready object built in <code>view_models.py</code> for one page. Templates read only view models
Workspace	The object a validator receives, whose reads and writes are confined to the registered roots after symbolic links are resolved, and which exposes the attempt's own evidence files
XP	Experience points declared per quest (<code>xp</code>), shown as two separate totals: claimed and verified

Golden Thread Quest system design, version 2.2.0. Rendered from docs/design/ by
scripts/build_design_html.py.